

Umgang mit dem Kursmaterial

- Sämtliches Kursmaterial (Texte, Folien, Abbildungen, Aufgaben, Lösungen usw.) unterliegt dem Urheberrecht des Dozenten und/oder weiterer Rechteinhaber.
- Die Nutzung ist ausschließlich den Studierenden der angegebenen Fachhochschule im Rahmen des angegebenen Kurses und nur zu Ausbildungszwecken gestattet.
- Jede weitergehende Nutzung, insbesondere das Veröffentlichen, Vervielfältigen, Verbreiten, Teilen oder öffentliche Zugänglichmachen des Materials (z.B. auf Webseiten, in Cloud-Speichern, per E-Mail, in Präsentationen oder Publikationen), ist ohne ausdrückliche schriftliche Zustimmung untersagt und kann rechtliche Konsequenzen nach sich ziehen.
- Die Nutzung des Kursmaterials in Verbindung mit Sprachmodellen oder KI-Systemen (z.B. LLMs wie ChatGPT) ist nur dann zulässig, wenn sichergestellt ist, dass die Inhalte nicht zum Training, zur Speicherung oder zu einer sonstigen Weiterverarbeitung durch Dritte verwendet werden (wie z.B. dem LLM-Anbieter) und alle oben genannten Nutzungsbeschränkungen eingehalten werden.



Kapitel_09

Speicherverwaltung

Betriebssysteme

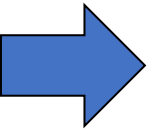
Prof. Dr. Claus Fühner

Fakultät Informatik

Ostfalia Hochschule für angewandte Wissenschaften

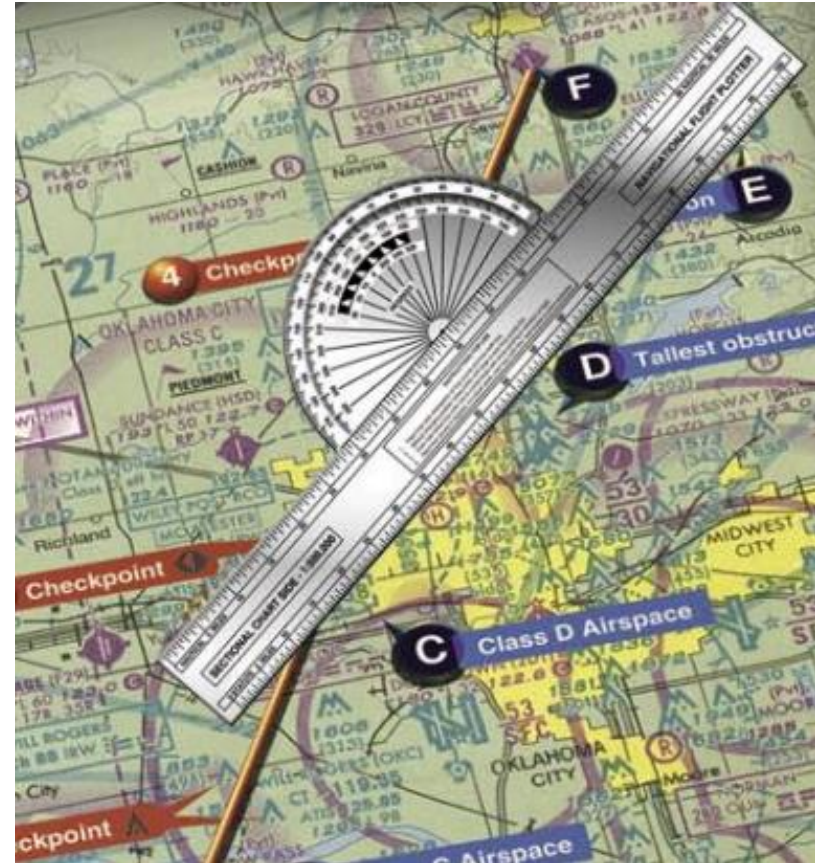
Braunschweig/Wolfenbüttel

Speicherverwaltung



Problemstellung

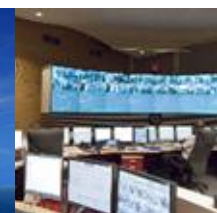
- Freier Speicher auf Prozessebene
- Virtueller Speicher
 - Segmentierung
 - Swapping
 - Paging
 - Seitenersetzungsstrategien
- Reales System



Nur zu Lehrzwecken



Betriebssysteme





Wunsch und Realität

Anforderung 1:

Jeder Programmierer hätte am liebsten

- (1a) einen unendlich großen und
- (1b) unendlich schnellen Speicher,
- (1c) der auch noch persistent ist.

Anforderung 2:

Speicher muss kostengünstig sein, damit Computer, Handys etc. preisgünstig angeboten werden können.

Wünsche für eine Speicherverwaltung:

- einem Prozess bzw. dem Gesamtsystem **mehr (virtuellen) Hauptspeicher** bieten **als physikalisch vorhanden**
- leichte **Relokation** von Programmen (Anpassung von absoluten Adressen bei Sprüngen und sonstigen Speicherzugriffen)
- Dynamisches Wachsen und Schrumpfen zur Laufzeit bei **Änderung des Speicherbedarfes** eines Prozesses
- **Gegenseitiger Speicherschutz** von Prozessen

Ebenen der Speicherverwaltung

Ebene 1: Betriebssystem

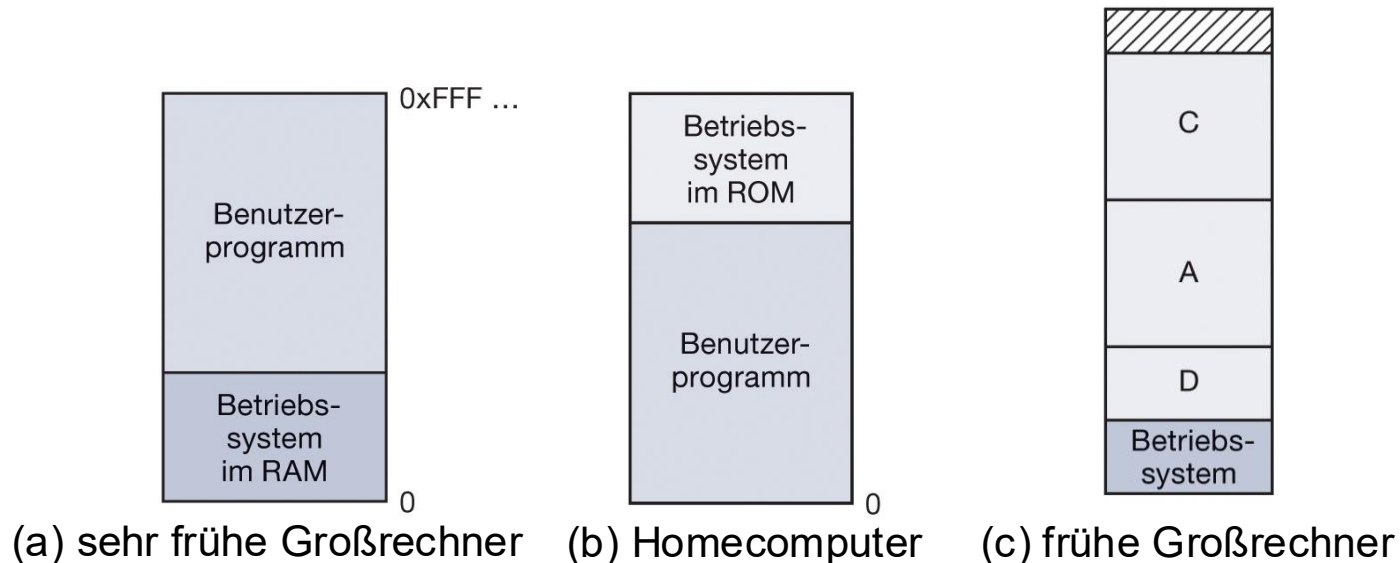
Wie wird der Speicher den unterschiedlichen Prozessen zugeteilt?

Ebene 2: Prozess

Wie wird der Speicher eines Prozesses intern verwaltet (→ Heap)?

(ähnlich: Wie wird der Speicher auf einer [Festplatte/SSD](#) verwaltet?)

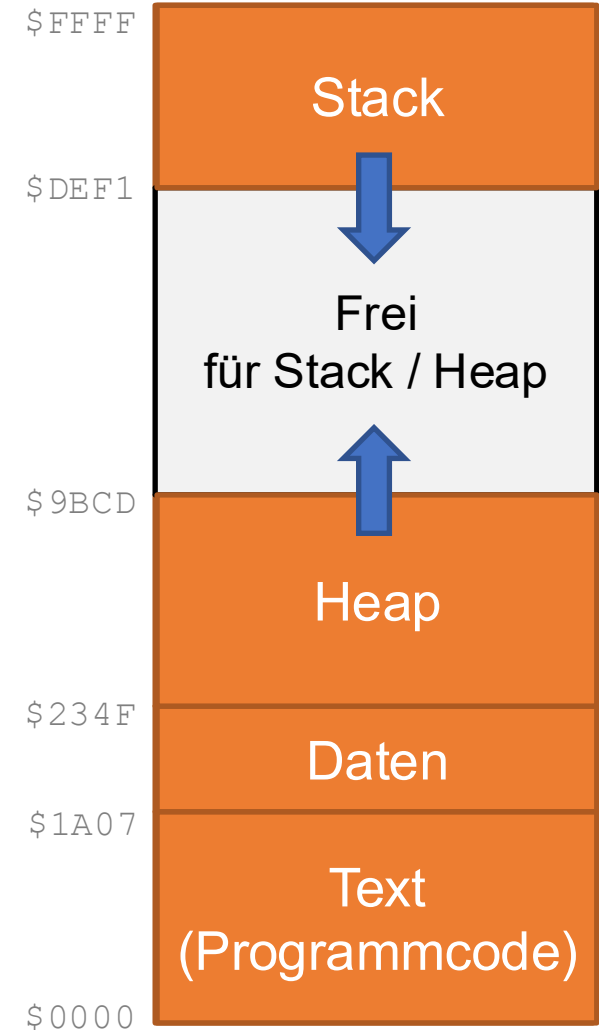
Speicherverwaltung in frühen Systemen ohne Speicherabstraktion



- anfangs **nur ein Prozess**, keine Parallelität
- variable Startadressen der Programme
→ **Relokationsproblem für absolute Adresse (globale Variablen, Sprünge)**
- fehlerhafte Speicherzugriffe eines Prozesses (zer-)stören das Betriebssystem oder andere Prozesse: **inhärent instabil**
- Bei mehreren mehreren Prozesse - **Wachsen** von A erfordert Verschieben von C: **ineffizient**
- Abhilfe: Speicherabstraktion mit Hilfe spezieller Hardware und Betriebssysteme

Prozess-interne Speicherverwaltung

- Heap: frei verwendbarer Speicherpool innerhalb eines Prozesses
- Ein Programm kann ...
 - ... Speicherblöcke anfordern:
new, malloc, ... (je nach Programmiersprache)
 - ... Speicherblöcke freigeben - diese stehen für eine spätere Anforderung zur Verfügung:
free, delete, dispose, ...
- Besonderheit Java/C#: automatische Freigabe durch Garbage Collector (Laufzeitumgebung)
- Beispiel: 2x malloc, 1x free, 1x malloc



Speicherverwaltung mit der C-Bibliothek

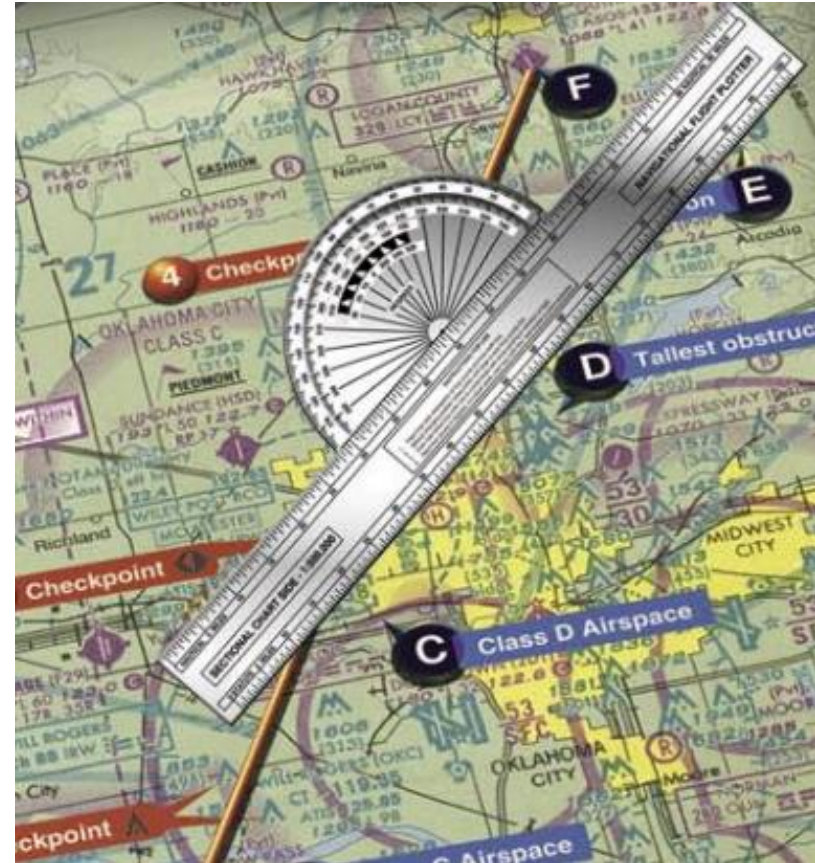
- `malloc()` / `free()` sind **Bibliothekaufrufe**
 - Selbst **keine Betriebssystemfunktionen**, nutzen aber Betriebssystemfunktionen **`brk()` / `sbrk()`**, um die Größe des Heap-Speichers anzupassen
 - Mit der Betriebssystemfunktion **`mmap()`** können neue virtuelle Speicherbereiche geschaffen werden.
- **Zahlreiche Fehlermöglichkeiten** bei Nutzung von `malloc()` / `free()`
 - Vergessen, Speicher zu allozieren
 - Zu wenig Speicher alloziert (-> Buffer overflow)
 - Initialisierung des Speicher nach `malloc()` vergessen
 - Freigabe von nicht mehr genutztem Speicher vergessen
 - Zu frühe Freigabe von Speicher, der noch genutzt wird
 - Wiederholte Freigabe von Speicher
 - Aufruf von `free()` mit falschem Parameter (kein durch `malloc()` erzeugter Pointer)

Agenda



Speicherverwaltung

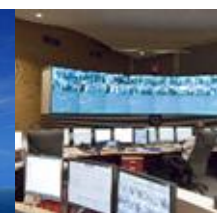
- Problemstellung
- ➔ Freier Speicher auf Prozessebene
- Virtueller Speicher
 - Segmentierung
 - Swapping
 - Paging
 - Seitenersetzungsstrategien
- Reales System



Nur zu Lehrzwecken



Betriebssysteme



Bitmap

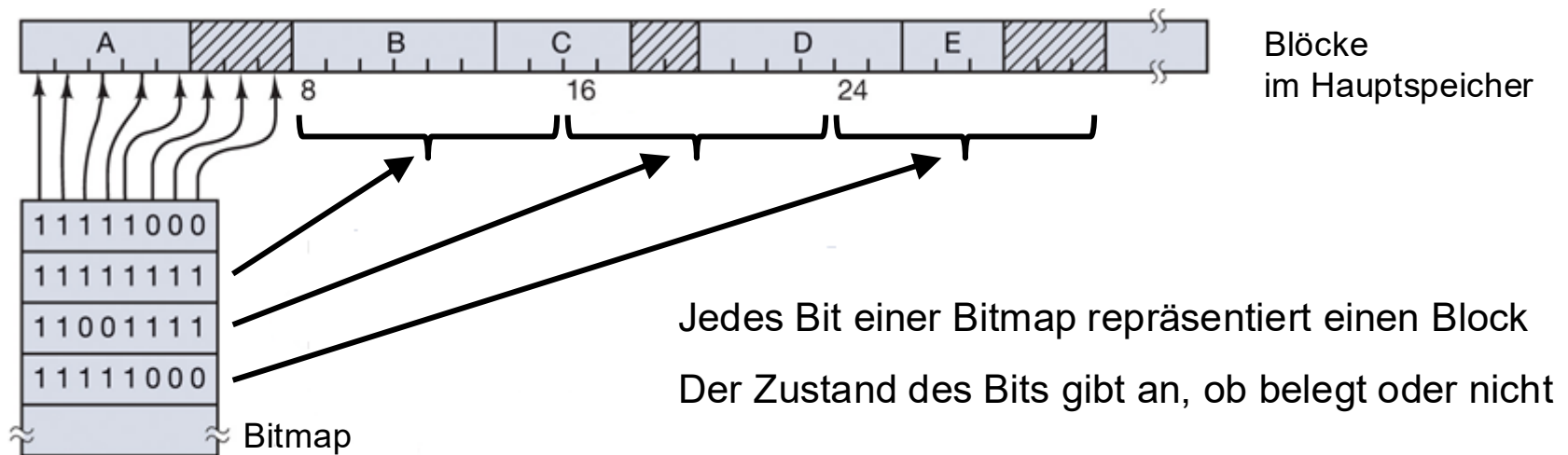
- Speicherverwaltung muss freie Bereiche im Hauptspeicher kennen, um diese bei Anforderung zuweisen zu können
- Idee 1: Einteilung des Speichers in **Belegungseinheiten (Blöcke)**, Verwaltung durch Bitmap

Blockgröße = _____



Bitmap

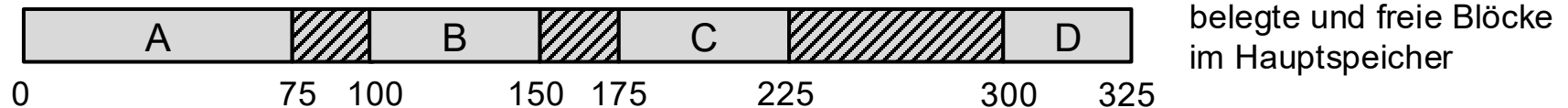
- Speicherverwaltung muss freie Bereiche im Hauptspeicher kennen, um diese bei Anforderung zuweisen zu können
- Idee 1: Einteilung des Speichers in **Belegungseinheiten (Blöcke)**, Verwaltung durch Bitmap



- **interne Fragmentierung** (Verschnitt durch Blockgröße) und **externe Fragmentierung** (genug Speicher frei, aber nicht „am Stück“)
- **Trade-off**: große Bitmap (kleine Blöcke) oder große interne Fragmentierung

Freibereichslisten

- Idee 2: Verkettung von Speicherblöcken beliebiger (dynamischer) Größe
- Beispiel: Verkettung der freien und belegten Speicherblöcke

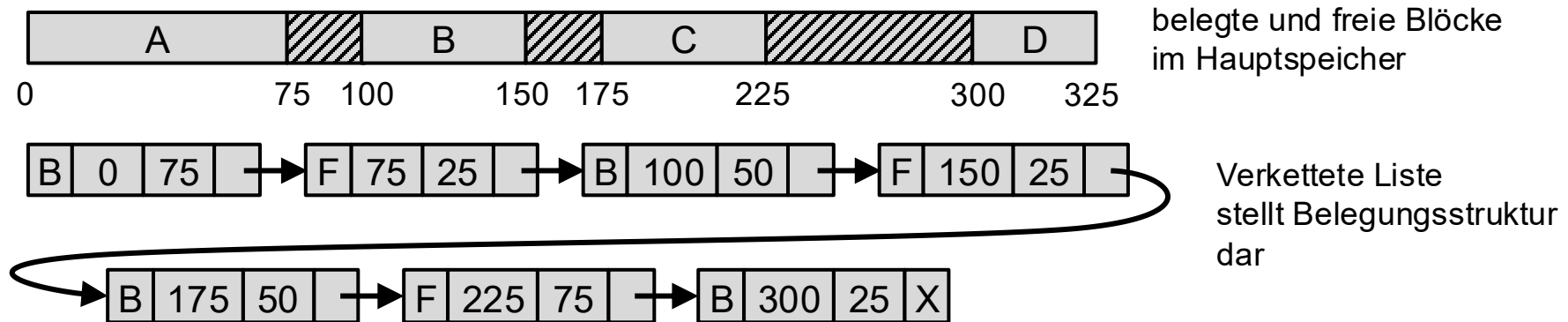


- Interne Fragmentierung?
- Externe Fragmentierung?
- In der Praxis werden meist nur die freien Speicherblöcke in der Speicherverwaltung erfasst



Freibereichslisten

- Idee 2: Verkettung von Speicherblöcken beliebiger (dynamischer) Größe
- Beispiel: Verkettung der freien und belegten Speicherblöcke



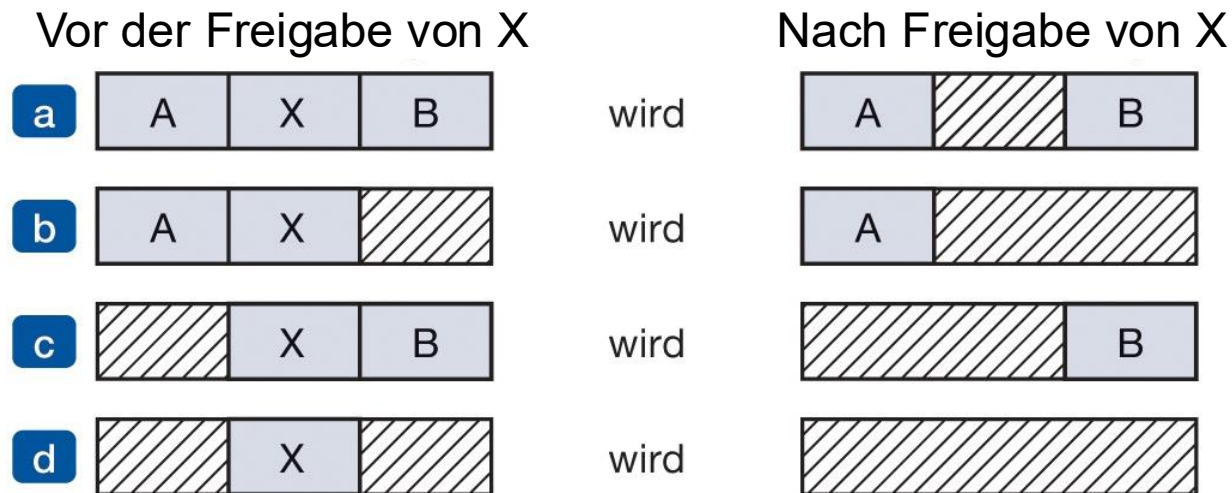
Optimierungen

- Liste nur für freie Speicherblöcke (die belegten Blöcke kennt ja das Programm)
- Freie Blöcke nutzen, um diese Liste zu speichern (-> minimale Blockgröße)
- Freie Blöcke aufteilen & zusammenfassen (splitting & coalescing) (nächste Folie)
- Strategie für Lückenwahl (später)

Freibereichslisten: Aufteilen & Zusammenfassen

Szenarien

- Drei Speicherblöcke A, X und B (→ Abbildung)
- Zuerst kann A und/oder B freigegeben werden, dann X



- Zusammenfassung des freigegebenen Blocks mit vorhergehendem / nächstem / beiden Blöcken, wenn diese frei sind

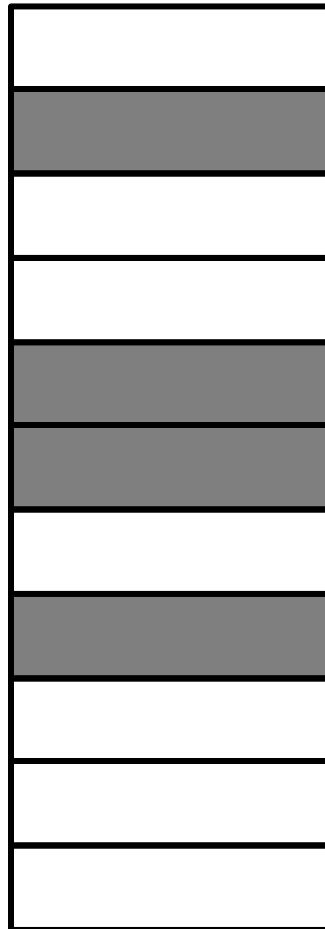
new/malloc: Auswahl des Speicherbereichs

- Strategien für die Reservierung eines neuen Speicherbereichs
 - First Fit – erste passende (groß genug) Lücke wird verwendet
 - Next Fit – nächste passende Lücke (ausgehend von der zuletzt genutzten)
 - ^{//}Best Fit – kleinste passende Lücke wählen (geringster Verschchnitt)
 - ^{//}Worst Fit – immer die größte Lücke wählen (nicht so viele kleine Lücken)

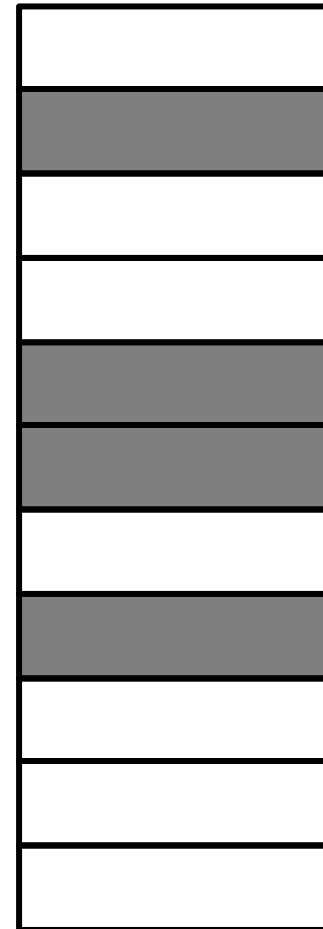
new/malloc: Auswahl des Speicherbereichs



First Fit



Next Fit



Best Fit



Worst Fit

new/malloc: Auswahl des Speicherbereichs

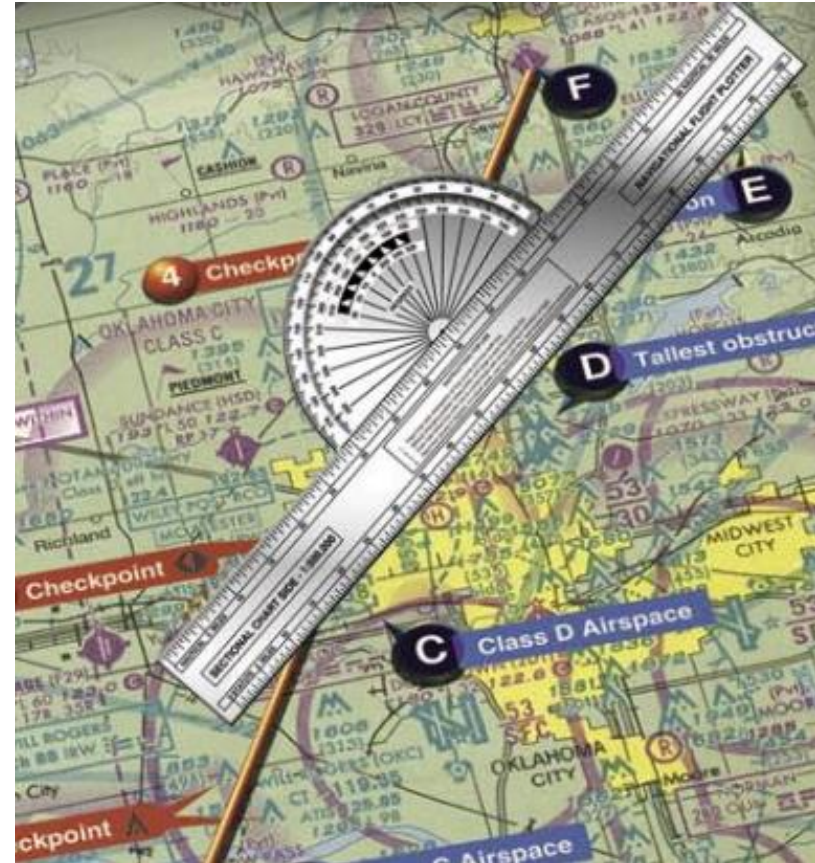
- Strategien für die Reservierung eines neuen Speicherbereichs
 - **First Fit** – erste passende (groß genug) Lücke wird verwendet
 - **Next Fit** – nächste passende Lücke (ausgehend von der zuletzt genutzten)
 - **Best Fit** – kleinste passende Lücke wählen (geringster Verschchnitt)
 - **Worst Fit** – immer die größte Lücke wählen (nicht so viele kleine Lücken)
- Bewertung
 - **Best Fit / Worst Fit** sind langsam, B.F. führt zu vielen sehr kleinen Lücken
- Weitere Optimierungen:
 - Lücken nach Größe sortieren
→ optimal für Suche, aber Verschmelzung von Einheiten schwierig
 - **Quick Fit**: Lücken nach Größe klassifizieren und in verschiedenen Listen verwalten, Verfeinerung: **Buddy Algorithmus** (geht über BRN hinaus...)

Agenda



Speicherverwaltung

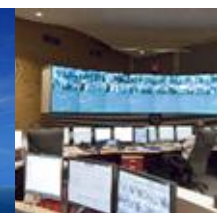
- Problemstellung
- Freier Speicher auf Prozessebene
- ➔ Virtueller Speicher
 - Segmentierung
 - Swapping
 - Paging
 - Seitenersetzungsstrategien
- Reales System



Nur zu Lehrzwecken

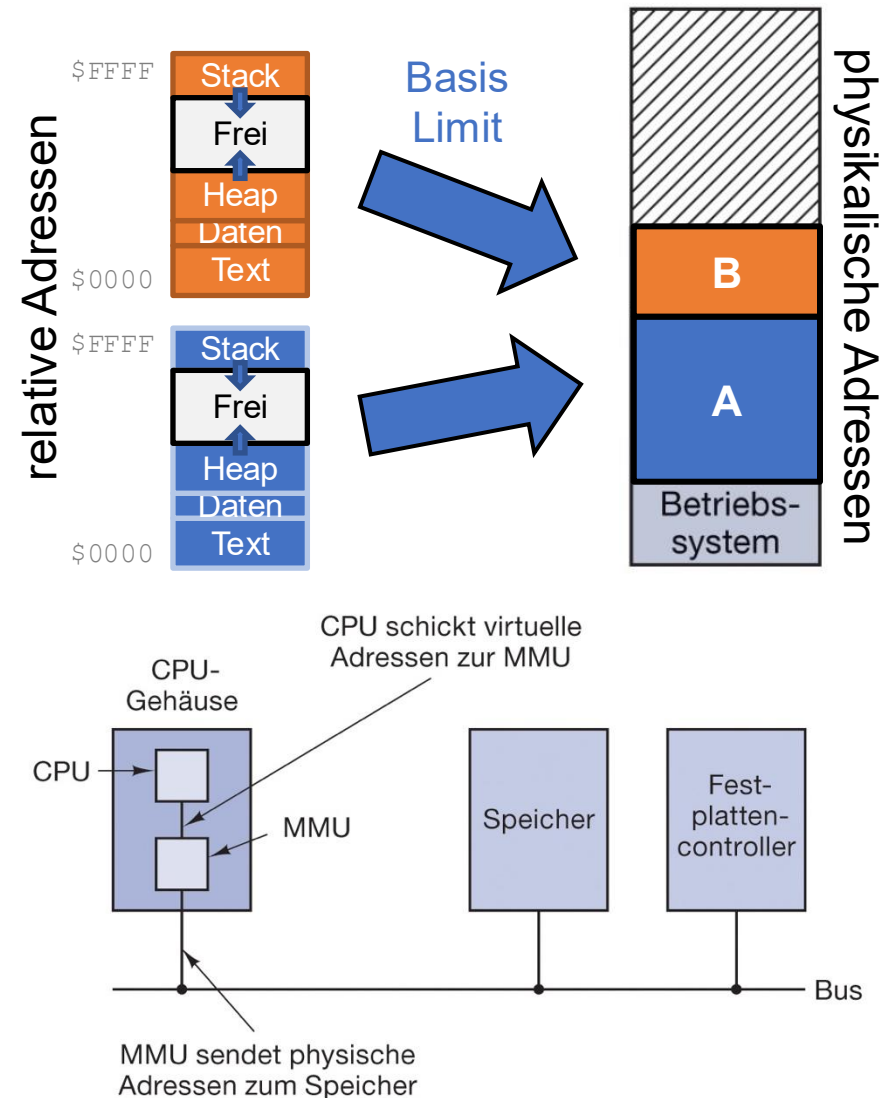


Betriebssysteme

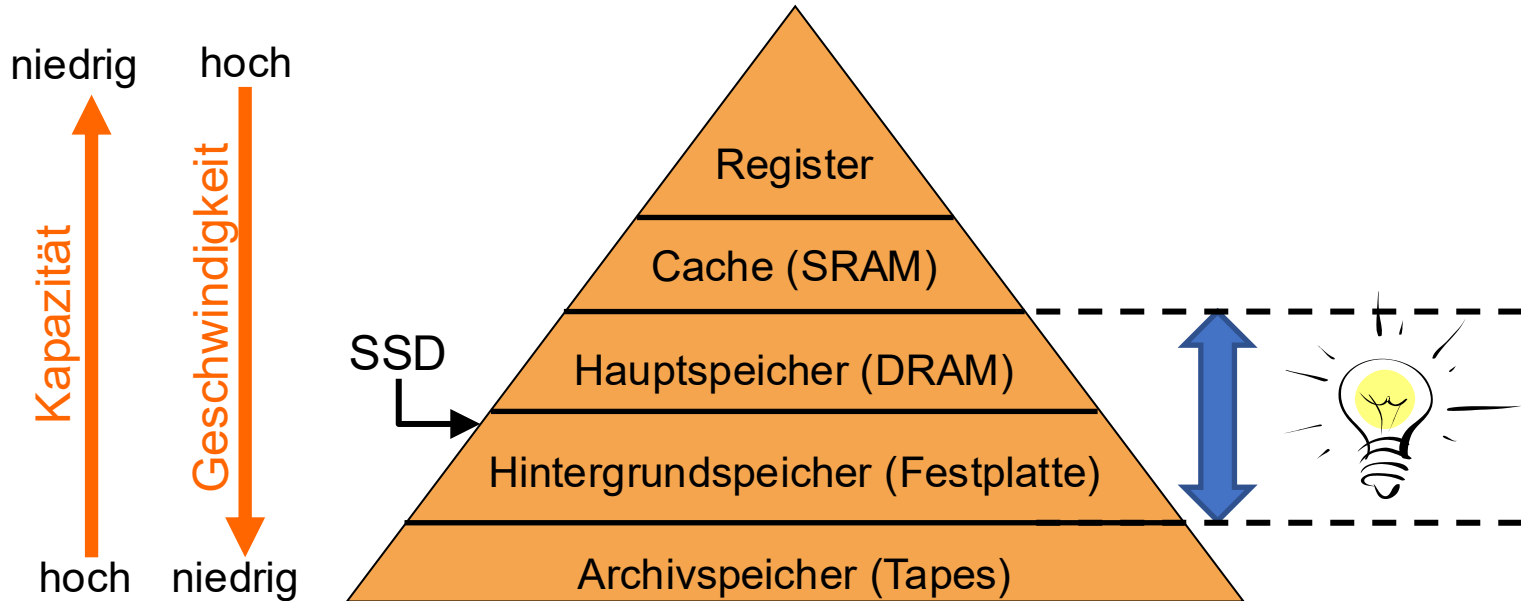


Übersicht Konzepte der Speicherabstraktion (Virtualisierung)

- Zunächst zusätzliche CPU-Register für **Basis**adresse und **Limit** (Länge) des Adressraums des aktiven Prozesses
- Programmcode mit **relativen Adressen**
- CPU berechnet physikalische Adressen durch Addition $Base + relAdr$
- Ermöglicht Relokation und Speicherschutz
- Später separate Speicher**segmente** für Code, Daten, Heap & Stack mit eigener Basis + Limit
- Dann **Paging** zur flexiblen Umsetzung virtueller in physikalische Adressen durch zusätzliche **Memory Management Unit (MMU)** basierend auf feingranularen Speicherseiten
- Virtualisierung oft mehrstufig (Segmentierung, Paging)



Virtueller Speicher und die Speicherhierarchie



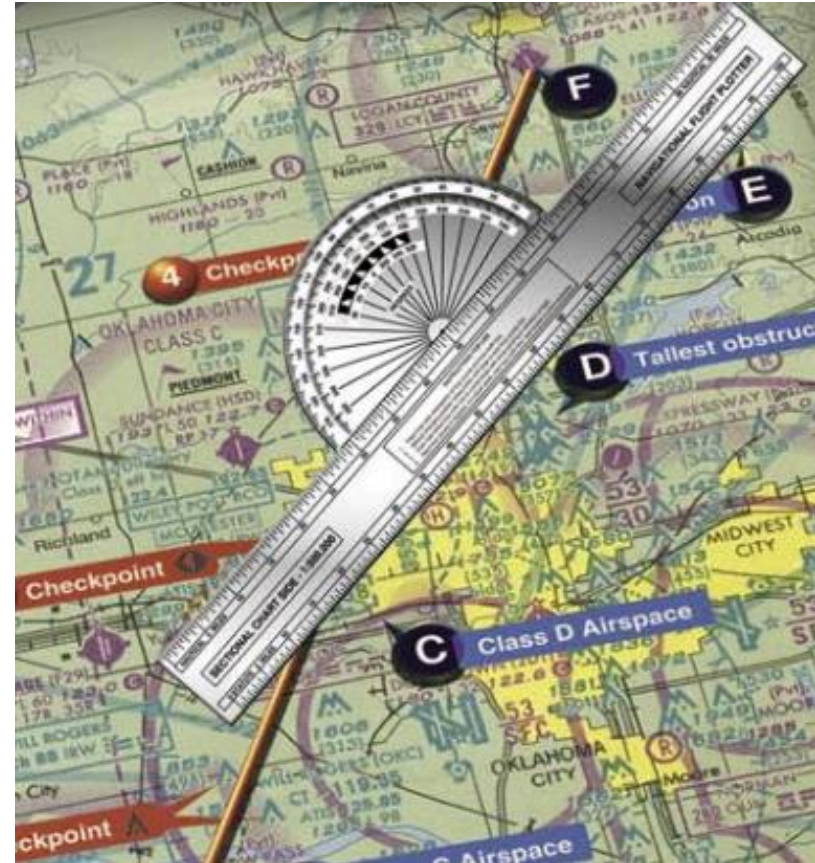
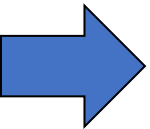
- SSDs stellen zusätzliche Stufe zwischen DRAM und Festplatte dar
- Virtuelle Adressräume aller Prozesse viel größer als physikal. Hauptspeicher
- Betriebssystem kann virtuellen Speicher auf Festplatte auslagern, um im Hauptspeicher Platz zu schaffen
- Betriebssystem lädt den Rest bei Bedarf von der Festplatte nach

Agenda



Speicherverwaltung

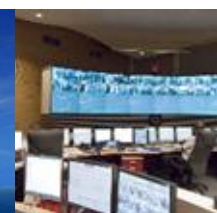
- Problemstellung
- Freier Speicher auf Prozessebene
- Virtueller Speicher
 - Segmentierung
 - Swapping
 - Paging
 - Seitenersetzungsstrategien
- Reales System



Nur zu Lehrzwecken



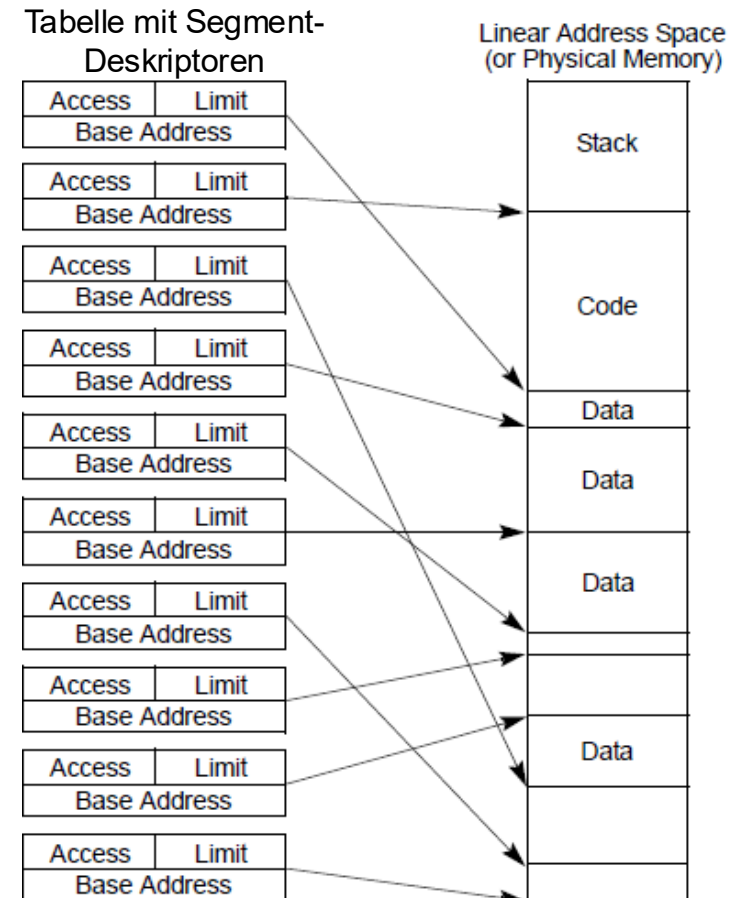
Betriebssysteme





Konzept

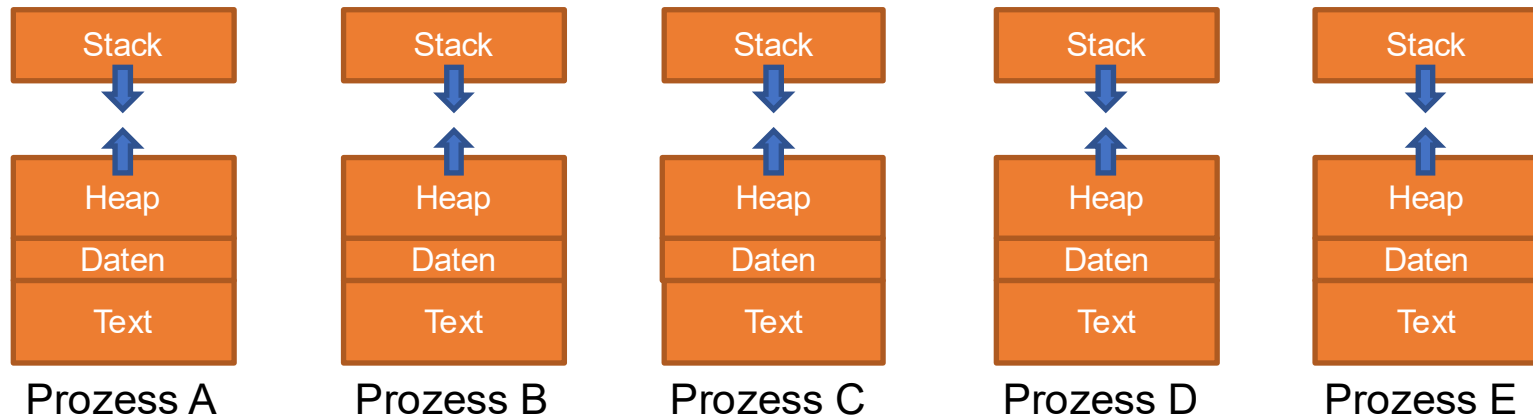
- Segmente: **Separate Adressbereiche** für jeden Speicherbereich (Text, Data, Heap, Stack) jedes Prozesses
- **Hardwareunterstützung** in CPU/MMU
- Konkretes Beispiel (ab 80286):
 - Tabelle mit **Segment-Deskriptoren**
 - Enthalten Basisadresse und Limit
 - CPU-Register für **Segmentnummern** (Indices)
 - **Hardware berechnet physikalische Adresse aus Segment-Nummer:Offset**
- **Isolation** von Prozessen und **Speicherschutz** mit Zugriffsrechten
- Betriebssystem schaltet Segmente beim Prozesswechsel um



aus: Intel 64 and IA-32 Architectures Software Developer's Manual (2013)



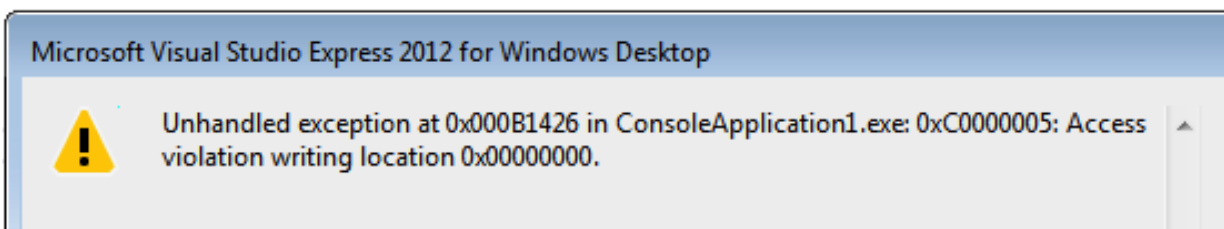
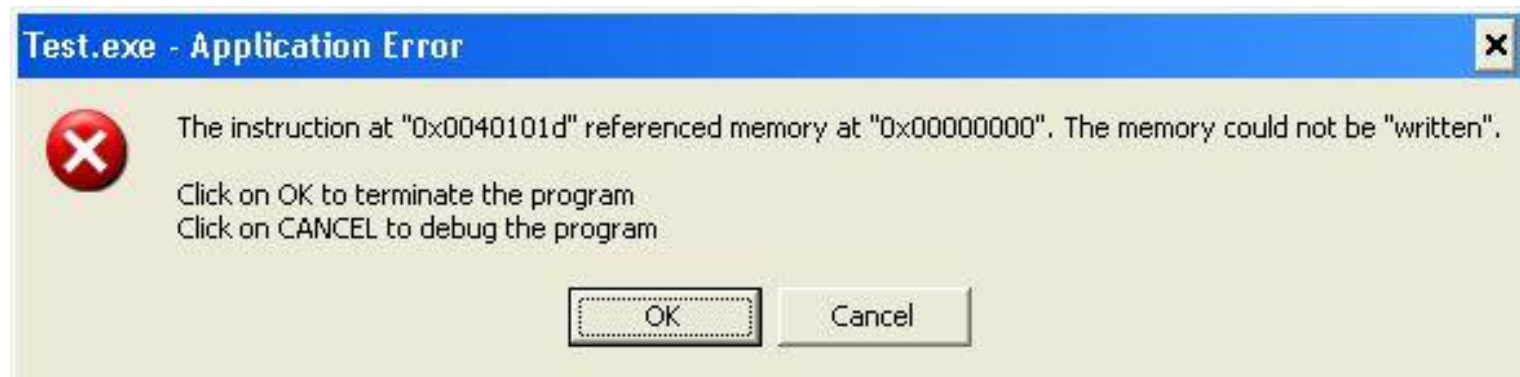
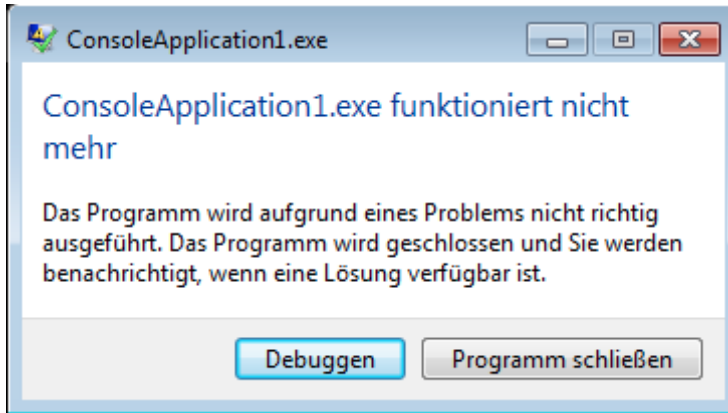
Segmentierung führt zu einem stabileren System



- **Kein Prozess** kann in ein Segment eines anderen Prozesses schreiben.
- Prozesse können meist **nicht ins Textsegment schreiben**, da dies den Programmcode korrumpieren würde.
- Prozesse können im Data-, Heap- und Stacksegment meist **keinen Code ausführen**, da diese Segmente nicht für Programmcode vorgesehen sind.
- **Fehlerhafte Speicherzugriffe** auf Offset > Segmentlänge führen zu Interrupts – die Programme werden dann vom Betriebssystem abgebrochen.



Segmentierung hilft, Programmierfehler aufzudecken

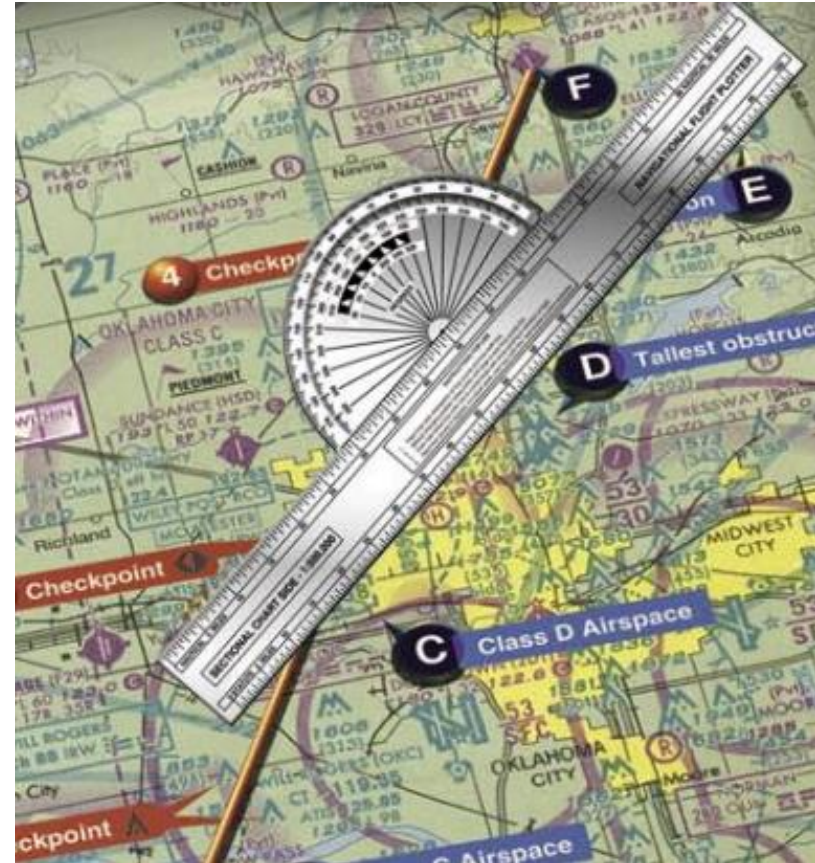
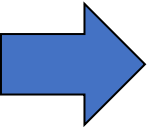


Agenda



Speicherverwaltung

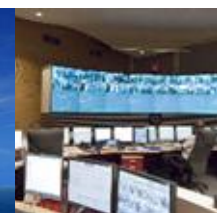
- Problemstellung
- Freier Speicher auf Prozessebene
- Virtueller Speicher
 - Segmentierung
 - Swapping
 - Paging
 - Seitenersetzungsstrategien
- Reales System



Nur zu Lehrzwecken



Betriebssysteme



Was können wir tun, wenn uns der Hauptspeicher ausgeht?

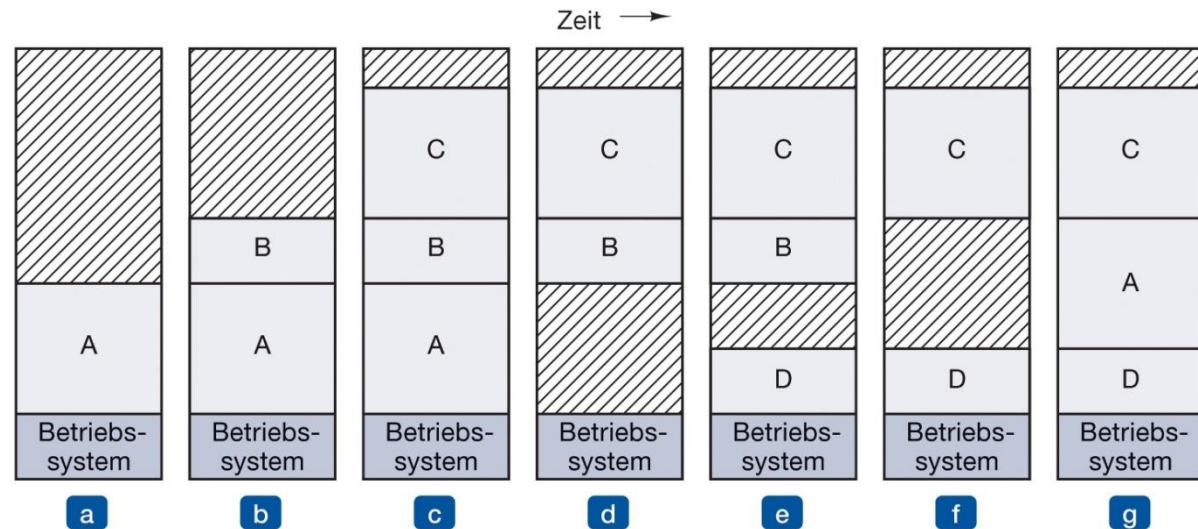
1. *Alle weiteren Speicheranforderungen fehlschlagen lassen*

- Start weiterer Prozesse nicht möglich
- Führt zu Fehlern in laufenden Programmen, wenn diese mehr Speicher anfordern
- ➔ keine ideale Lösung

2. *Speicher laufender Prozesse aus dem Hauptspeicher entfernen*

- Speicherinhalt könnte vor dem Entfernen auf der Platte zwischengespeichert werden
- Hauptspeicher kann dann anderweitig genutzt werden
- Spätere Restaurierung möglich
- **Ziel: Der physikalische Hauptspeicher soll für die Prozesse größer aussehen, als er tatsächlich ist!**
- Lösungsansätze:
 1. Swapping ganzer Segmente oder Prozesse
 2. Demand Paging

- **Z. B. komplette Prozesse** werden bei Leerlauf auf die Festplatte ausgelagert und erst bei Bedarf in den Hauptspeicher geladen.



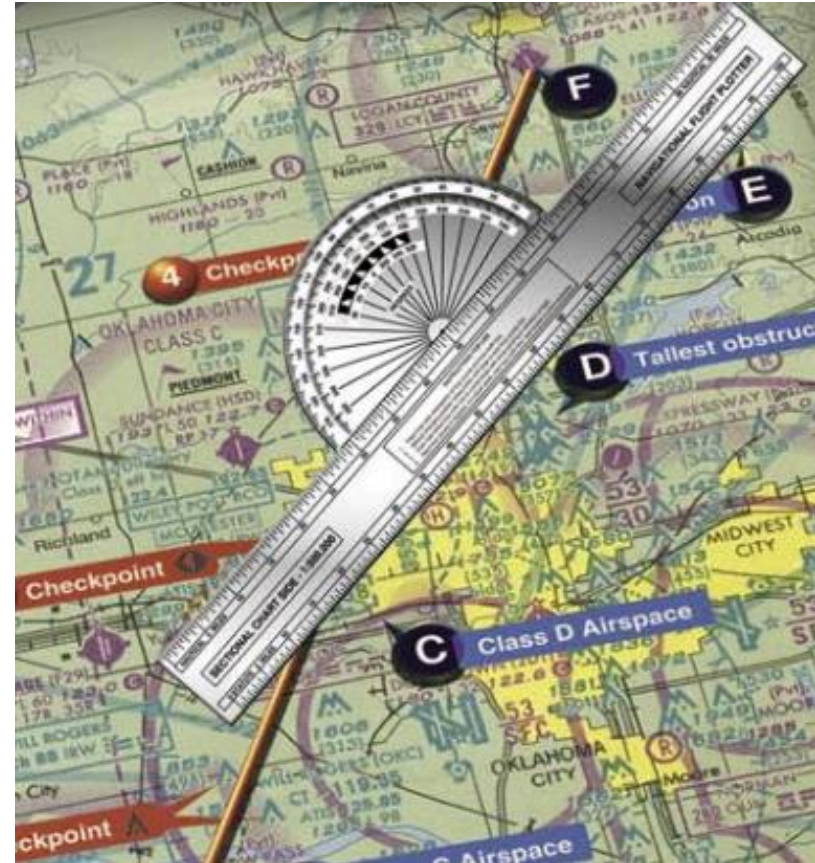
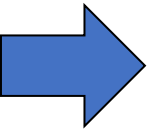
- **Probleme:**
 - Fragmentierung des Hauptspeichers
 - Prozesse sind nicht konstant im Speicherbedarf, sie können wachsen und wieder schrumpfen
 - Prozess muss komplett in den Speicher passen
 - Performanz beim Ein- und Auslagern (HD: 1 GB schreiben dauert ca. 10s)
- **Verbesserungsidee:** Kleinere Blöcke als ganze Segmente verwalten!

Agenda



Speicherverwaltung

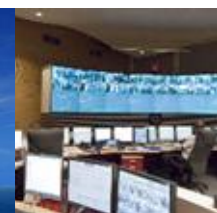
- Problemstellung
- Freier Speicher auf Prozessebene
- Virtueller Speicher
 - Segmentierung
 - Swapping
 - Paging
 - Seitenersetzungsstrategien
- Reales System



Nur zu Lehrzwecken



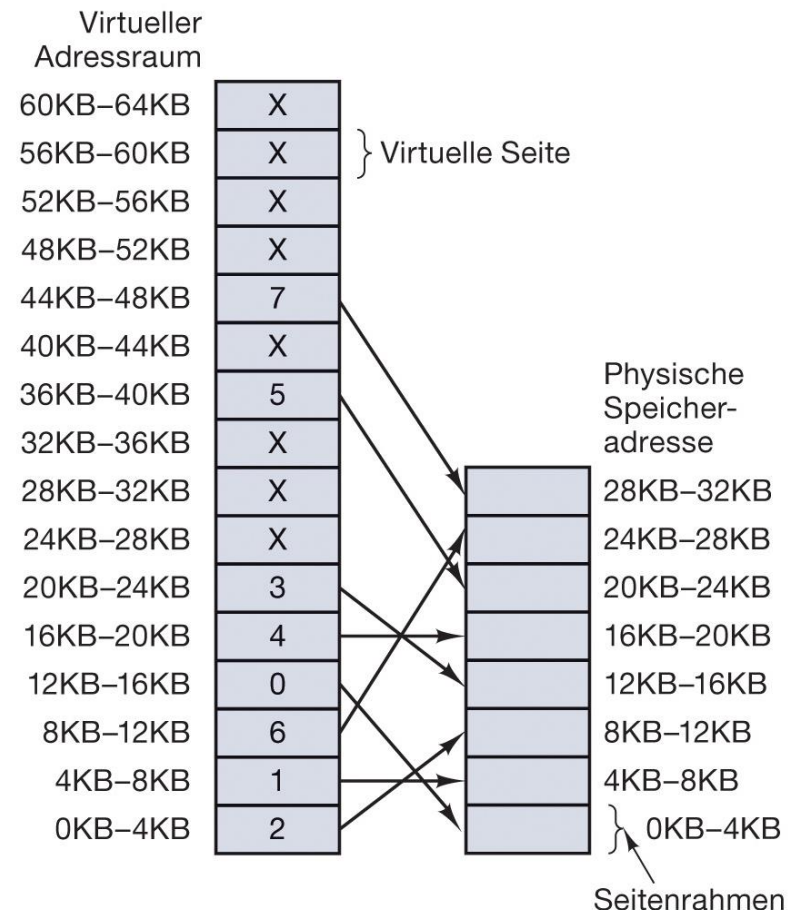
Betriebssysteme



Grundidee

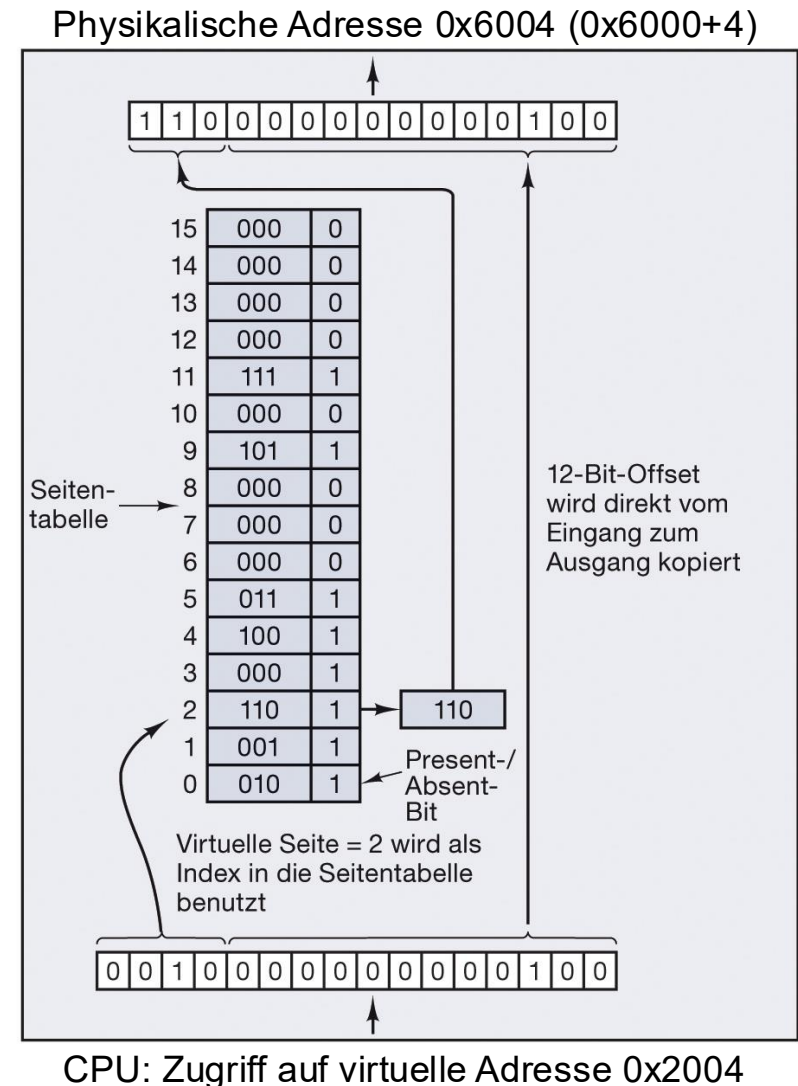
- **Virtueller Adressraum** eingeteilt in (relativ kleine) **Seiten (Pages)**
- **Physikalische Speicher** eingeteilt in **Seitenrahmen (Page Frames)** gleicher Größe
- Zuordnung von Seiten zu Seitenrahmen durch MMU mit **Seitentabelle (page table)**
- Zugriff auf (virtuelle) Seite, die sich nicht im Speicher befindet
 - ➔ MMU löst **Seitenfehler (page fault)** aus
 - ➔ **Betriebssystem** wählt z. B. eine zu ersetzende Seite aus, sichert diese und lädt die fehlende nach
- Umsetzung systemweit oder pro Prozess
- Paging hat Swapping meist ersetzt, Begriff *Swapping* meint in Unix heute meist *Paging*)

Beispiel der Zuordnung:



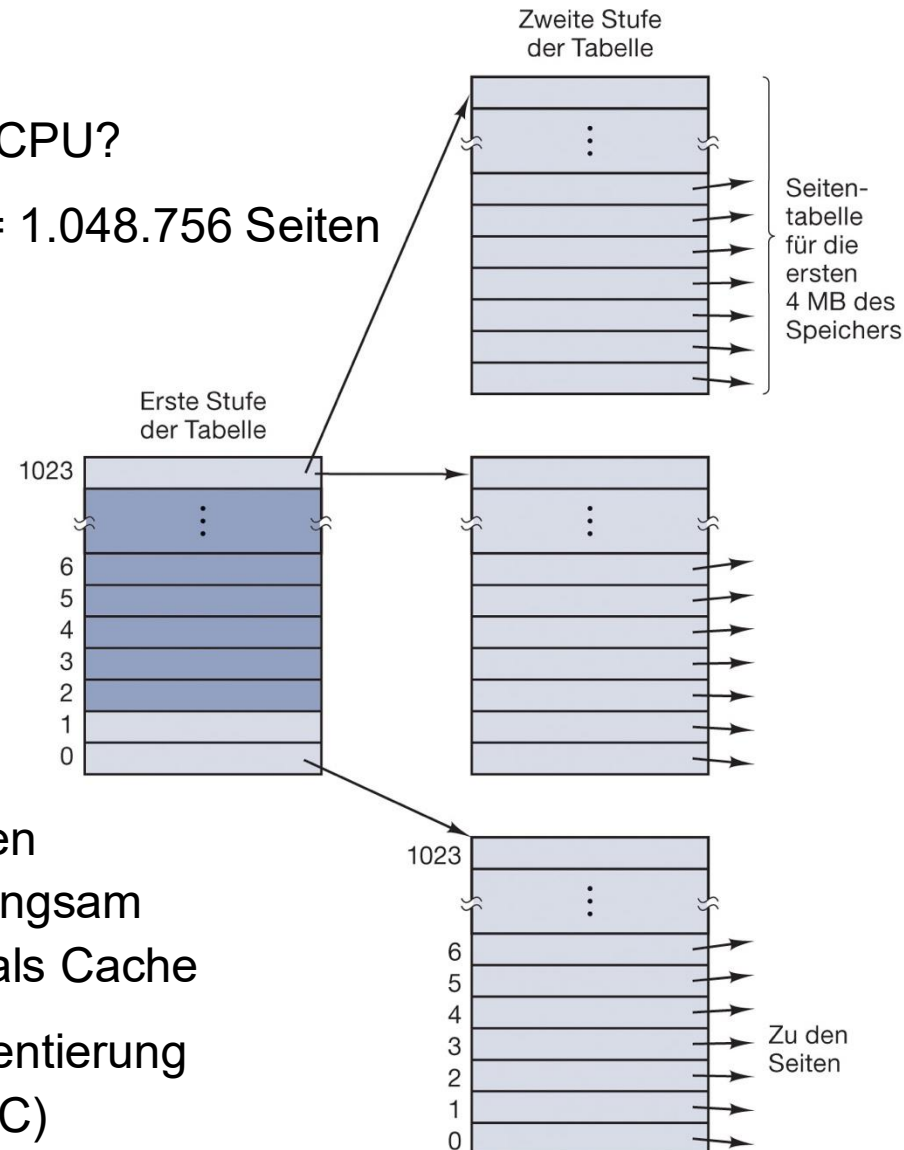
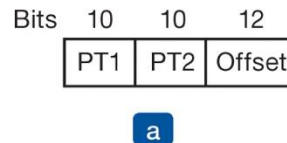
Beispiel für Paging durch MMU

- Virtuelle Adressen: 16 Bit
- Physikalische Adressen: 15 Bit
- Seitengröße: 4 kB (12 Bit)
- Seitentabelle liegt im Hauptspeicher
- Startadresse der Seitentabelle der MMU bekannt
- MMU nutzt die oberen 4 Bit einer virtuellen Adresse als Index in die Seitentabelle
- Seitentabelle enthält obere 3 Bit der physikalischen Adresse
- Untere 12 Bit der Physikalischen Adresse wie virtuelle Adresse
- Real oft mehrstufige Seitentabellen



Mehrstufige Seitentabellen

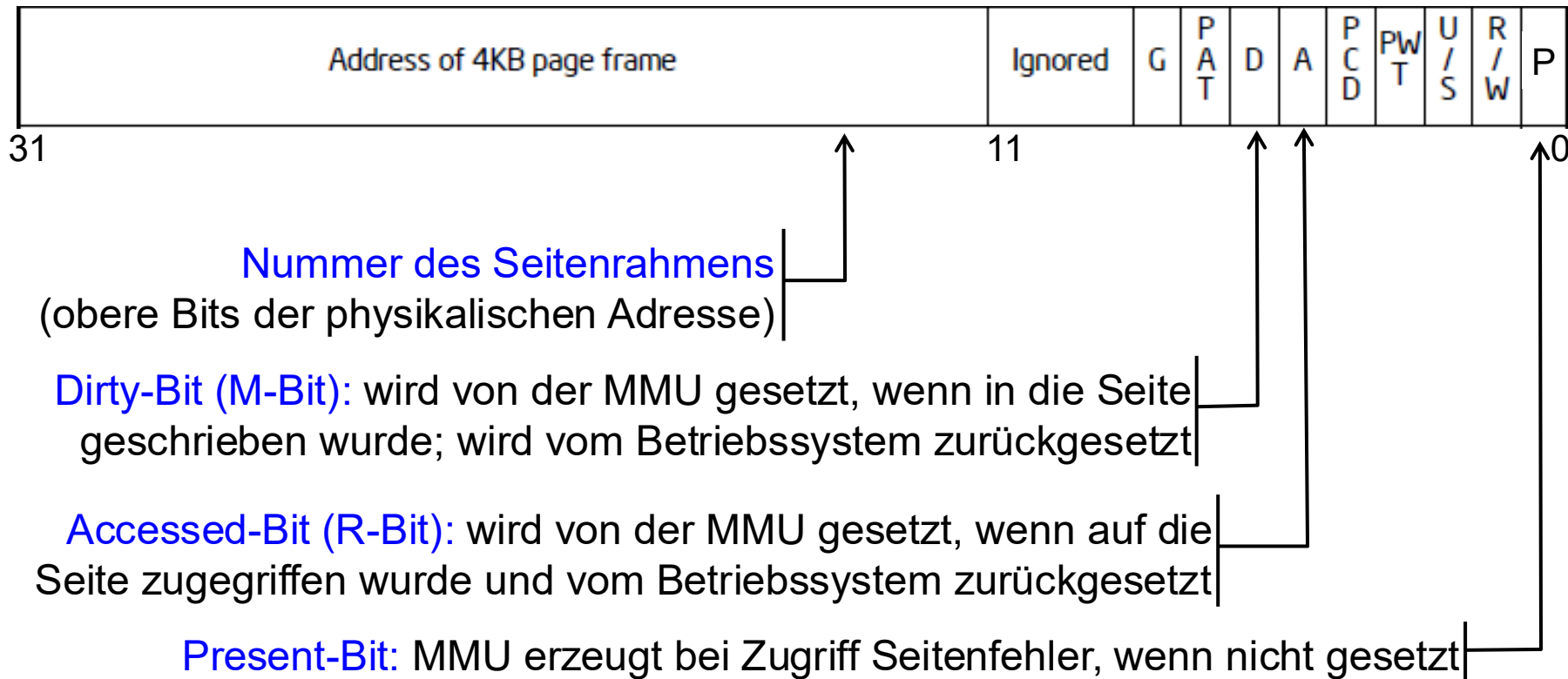
- Größe der Seitentabelle für eine 32 Bit CPU?
- Seitengröße 4 kiB erfordert 2^{20} Seiten = 1.048.756 Seiten
- Lösung: mehrstufige Seitentabellen
- Vorteil: Seitentabellen müssen nicht komplett im Speicher gehalten werden



- Nachteil: Bestimmung der physikalischen Speicheradresse wird aufwändig und langsam
→ **TLB (Translation Lookaside Buffer)** als Cache
- Großer TLB ermöglicht Paging-Implementierung in komplett Software (z.B. MIPS, SPARC)



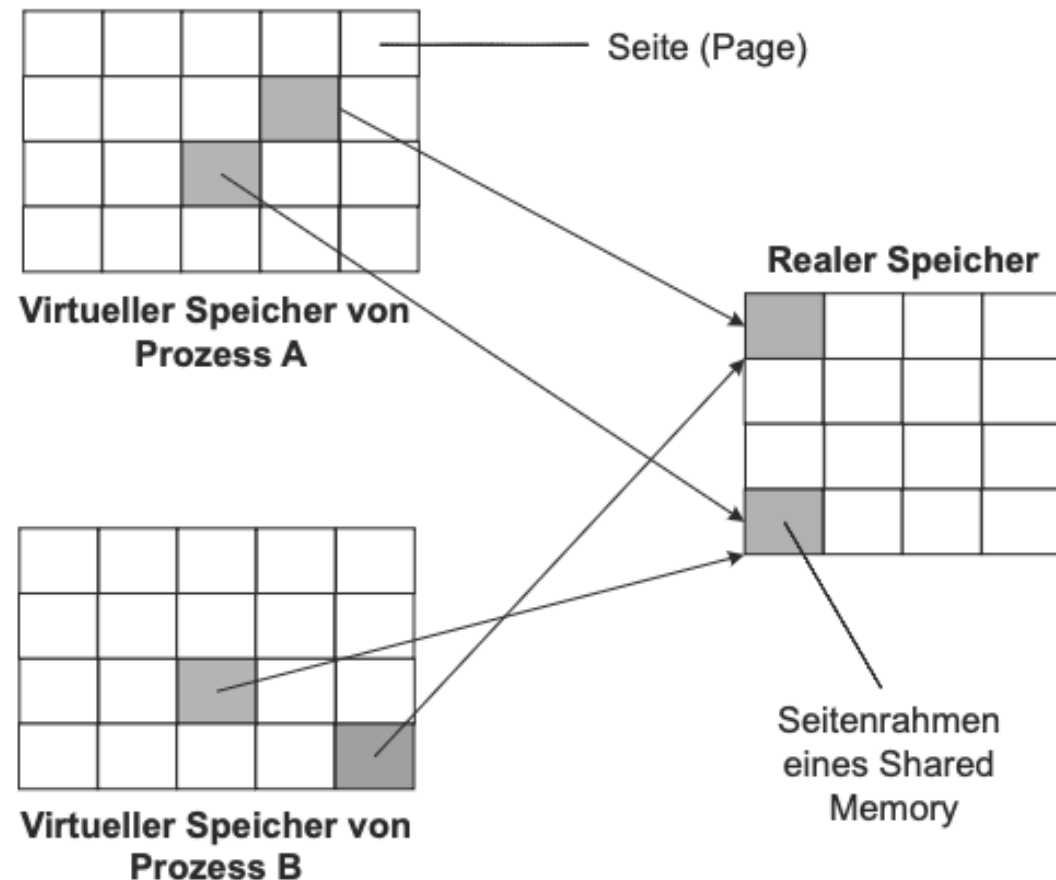
Realer Eintrag in Seitentabelle (IA-32)



Weitere Bits z. B. zur Steuerung von **Caching** und **Zugriffsrechte**

Shared Memory

- **Shared Memory:**
Verschiedene Prozesse teilen sich einen Speicherbereich
- Spart Hauptspeicher
- ermöglicht **sehr schnelle Interprozess-kommunikation (IPC)**, erfordert dann aber separate Synchronisation (Semaphore, Monitore, ...)
- Kann auch für Code in **Shared Libraries (Linux)** oder **DLLs (Windows)** verwendet werden

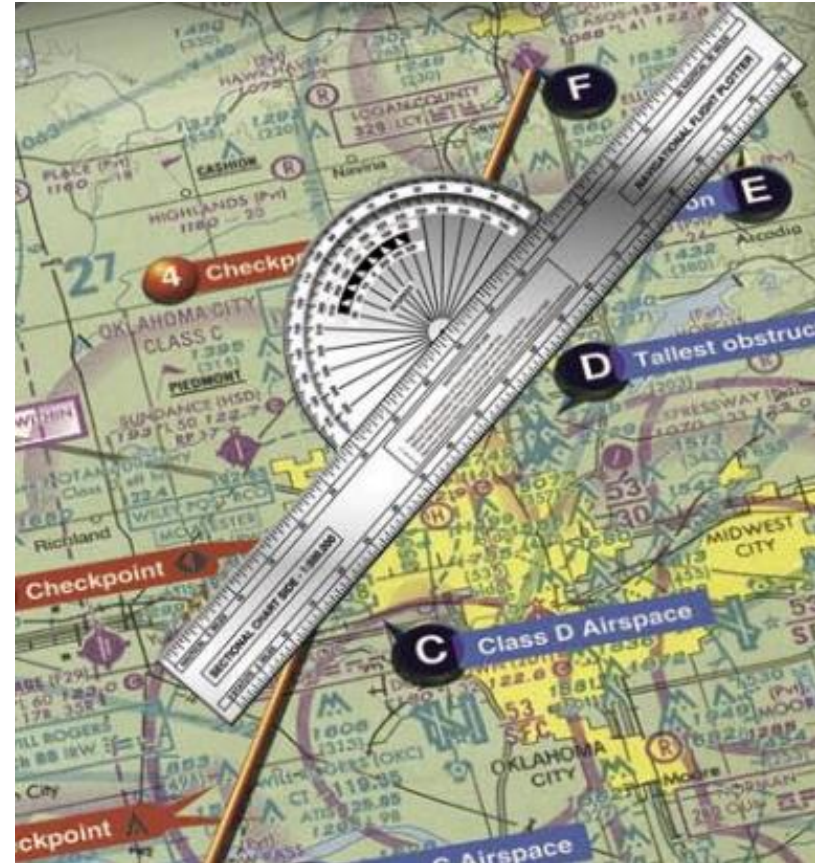
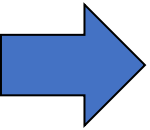


Agenda



Speicherverwaltung

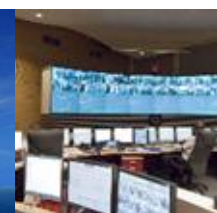
- Problemstellung
- Freier Speicher auf Prozessebene
- Virtueller Speicher
 - Segmentierung
 - Swapping
 - Paging
 - Seitenersetzungsstrategien
- Reales System



Nur zu Lehrzwecken



Betriebssysteme



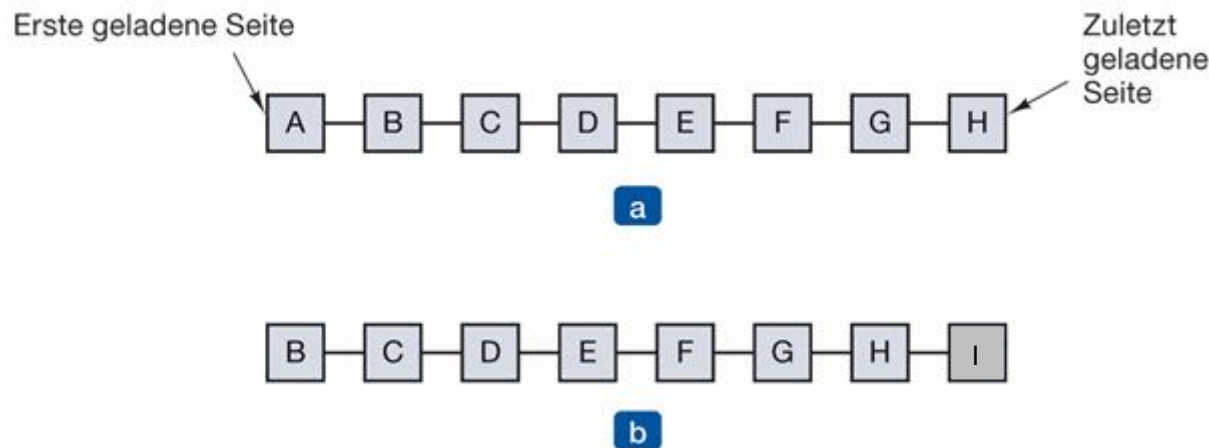
Seitenersetzungsalgorithmus (Page Replacement Alg.)

Wir benötigen Speicher und wollen eine Seite aus dem RAM auslagern, um einen Seitenrahmen freizuräumen. Welche Seite wählen wir aus?

- Falls die Seite modifiziert wurde, muss sie zuvor geschrieben werden, damit kein Datenverlust auftritt → *erhöhter Aufwand*
- Der **optimale Algorithmus** wählt die Seite, deren Aufruf am weitesten in der Zukunft liegt → *leider nicht praktisch bestimmbar*
- **Zufällige Auswahl** → *funktioniert überraschen gut*
- **FIFO? Least Recently Used? Weitere Optionen?**

First-In-First-Out-Algorithmus (FIFO)

- Die Seiten werden in einer verketteten Liste gespeichert.
- Am Ende der Liste wird immer die zuletzt geladene Seiten angehängt.
- Muss eine Seite ausgelagert werden, so wird immer die älteste Seite gewählt (am Beginn der Liste)

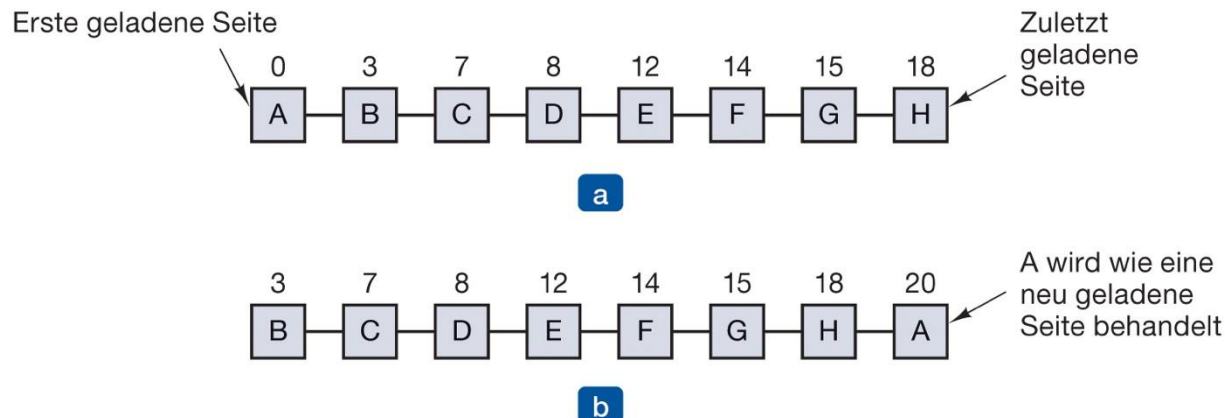


- Vorteile/Nachteile?

Second-Chance-Algorithmus (Clock Algorithmus)

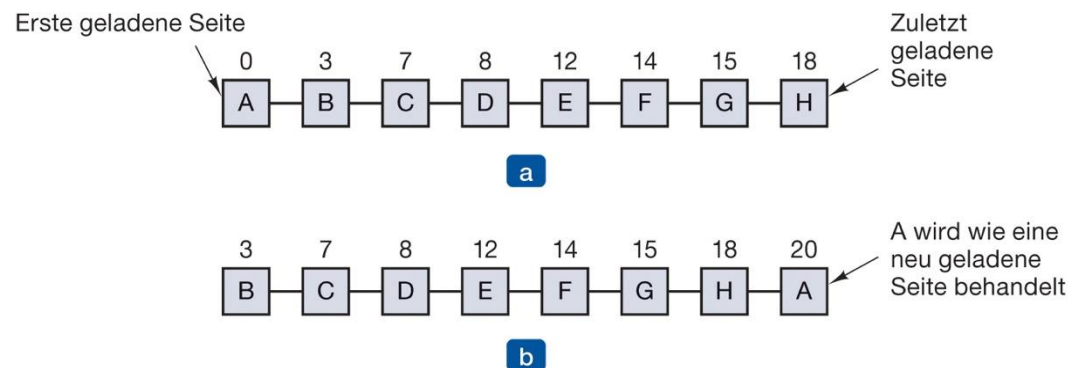
- Die geladenen Seiten werden in einer **FIFO- oder zirkulären Datenstruktur** gespeichert (die Liste ist damit nach den Einlagerungszeitpunkten sortiert)
- Bei **Zugriff** auf die Seite setzt die Hardware automatisch das R-Bit
- Die FIFO-Liste wird durchlaufen und das **R-Bit** geprüft
 - 0**: die Seite ist alt, seit gewisser Zeit unbenutzt und wird entfernt. Fertig.
 - 1**: Seite wird nicht sofort entfernt. R-Bit wird auf 0 gesetzt, der Zeitstempel wird aktualisiert und die Seite an das Ende der FIFO-Liste verschoben. Die Suche wird fortgesetzt.

(expliziter Zeitstempel nicht unbedingt notwendig, Listenposition entscheidend)



Least-Recently-Used-Algorithmus (LRU)

- **Beobachtung:**
 - Seiten, die kürzlich benötigt wurden, werden oft auch für die nächsten Befehle benötigt.
 - Seit längerem unbenutzte Seite bleiben häufig weiterhin unbenutzt.
- **Verfahren:**
 - Für jede Seite den Zeitstempel des letzten **Zugriffs** speichern
 - Die Seite, die am längsten unbenutzt ist, wird zur Auslagerung ausgewählt.



- Ideales LRU eher nicht genutzt, da technisch sehr aufwändig
- verschiedene Näherungen an LRU: → Tanenbaum, Moderne Betriebssysteme

Not-Recently-Used-Algorithmus (NRU)

- Laden einer Seite: BS setzt *R*- und *M-Bits* auf 0
- Zugriff auf Seite: MMU setzt *R*- und *M-Bit* bzw. nur *R-Bit* auf 1
- In bestimmten Zeitabständen setzt das BS die *R-Bits* aller Seiten auf 0
→ bei selten verwendeten Seiten steht das *R-Bit* häufig auf 0
- Bei einem Seitenfehler werden alle Seiten entsprechend *R*- und *M-Bit* in *vier Kategorien* aufgeteilt (R-Bit, M-Bit):
 - Klasse 0: nicht referenziert, nicht modifiziert
 - Klasse 1: nicht referenziert, modifiziert
 - Klasse 2: referenziert, nicht modifiziert
 - Klasse 3: referenziert, modifiziert
- Der NRU-Algorithmus wählt eine zufällige Seite aus der niedrigsten, nicht leeren Klasse.

Wie kann dieser Zustand entstehen?

Cleaning-Optimierung

Demand Cleaning

- Seite wird erst dann auf die Festplatte/SSD geschrieben, wenn eine neuer Seitenrahmen benötigt wird

Precleaning

- Präventives Schreiben modifizierter Seiten zu *geeigneten* Zeitpunkten

Page Buffering

- *Modified List* mit Seiten, die freigegeben werden können (z.B. nach NRU)
- Zyklisches Schreiben von Seiten aus der *Modified List* in Gruppen und Verschieben in die *Unmodified List* (Cache)
- Wenn neuer Seitenrahmen benötigt, einen aus der *Unmodified List* verwenden
- Heute implementierte Variante

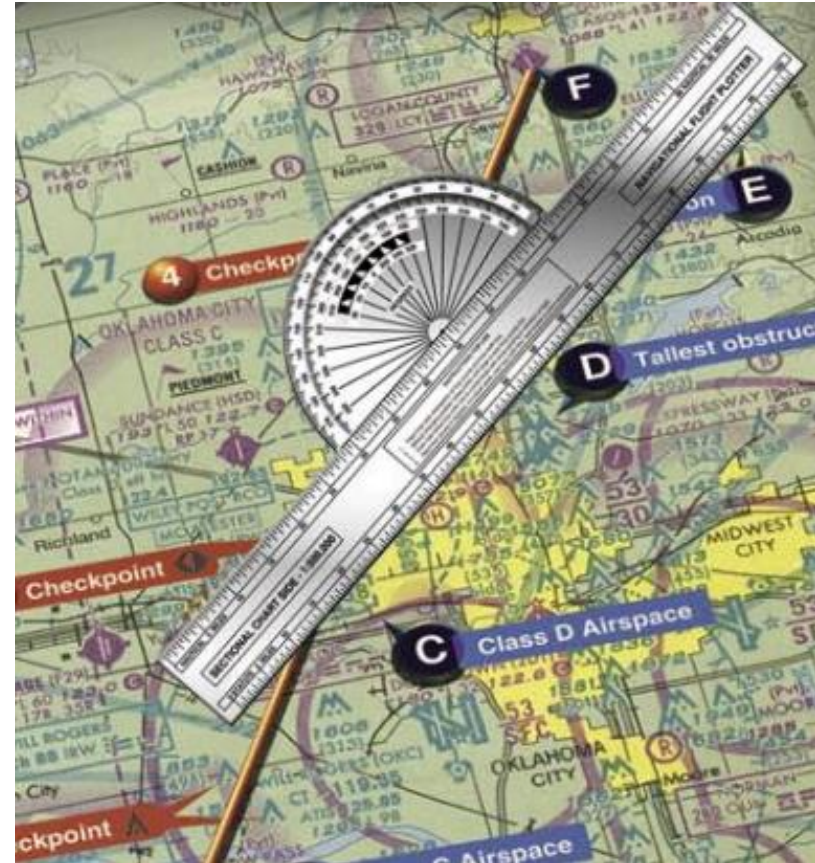
Agenda



Speicherverwaltung

- Problemstellung
- Freier Speicher auf Prozessebene
- Virtueller Speicher
 - Segmentierung
 - Swapping
 - Paging
 - Seitenersetzungsstrategien

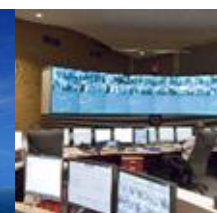
➔ Reales System



Nur zu Lehrzwecken



Betriebssysteme



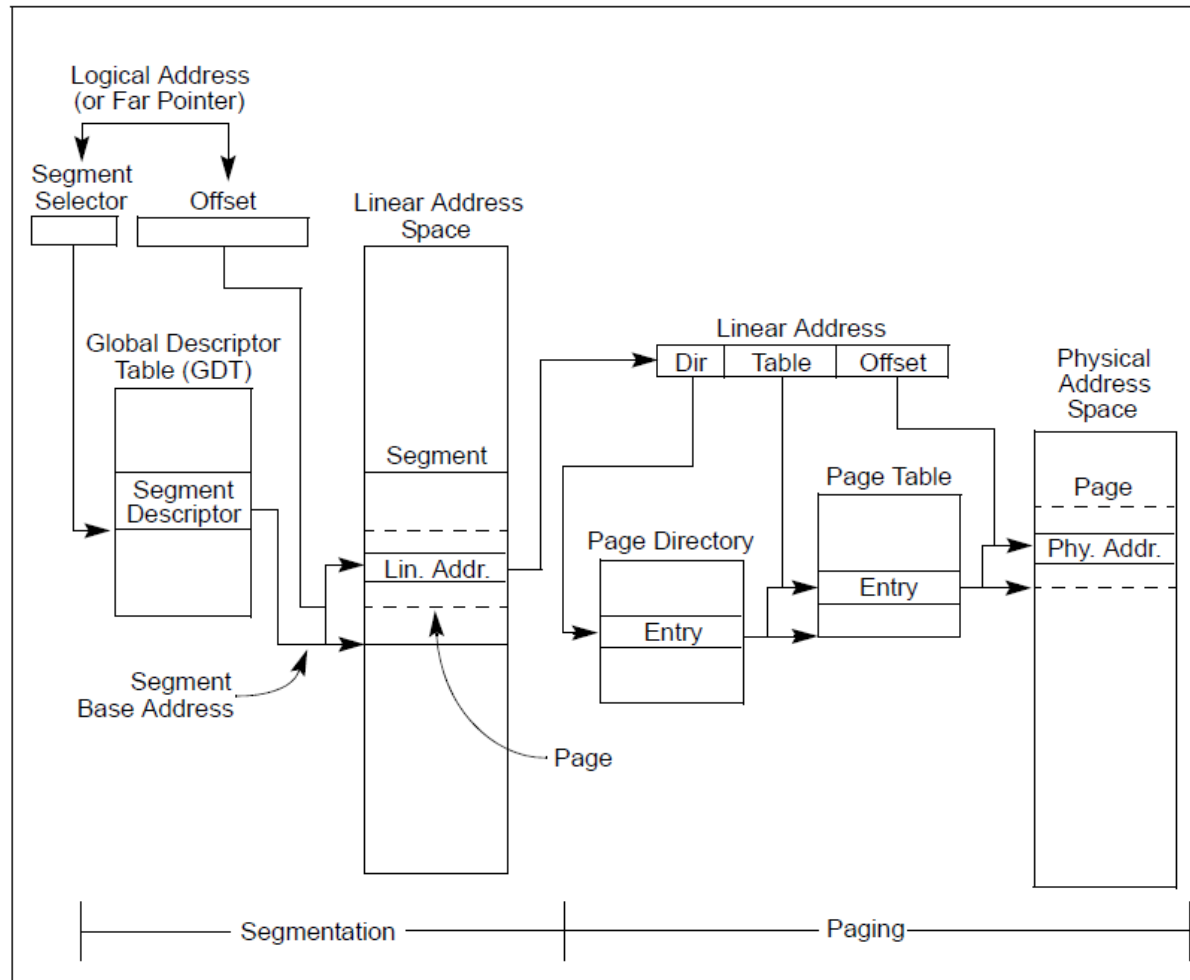
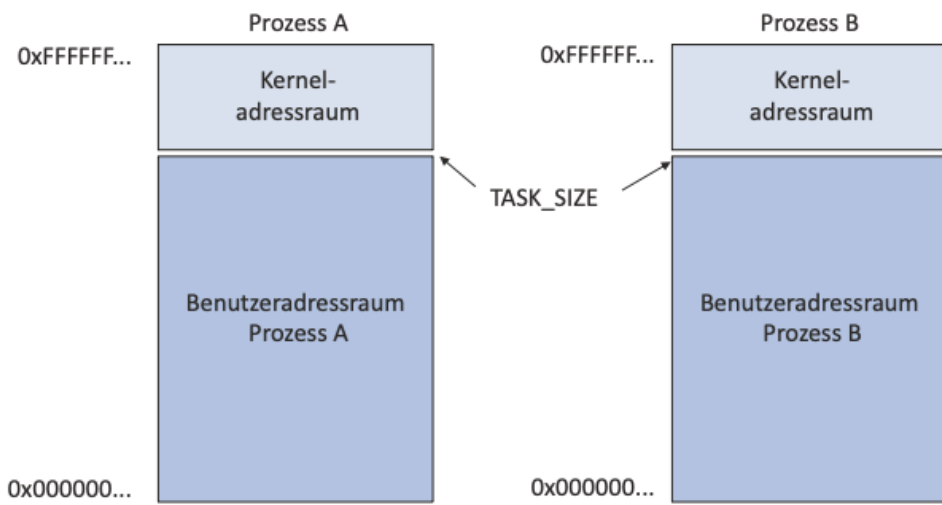


Figure 3-1. Segmentation and Paging

x86: Vollständige Realisierung in Hardware inkl.
Caching im TLB (Translation Lookaside Buffer)

aus: Intel 64 and IA-32 Architectures Software Developer's Manual (2013)

Speicherverwaltung in Linux

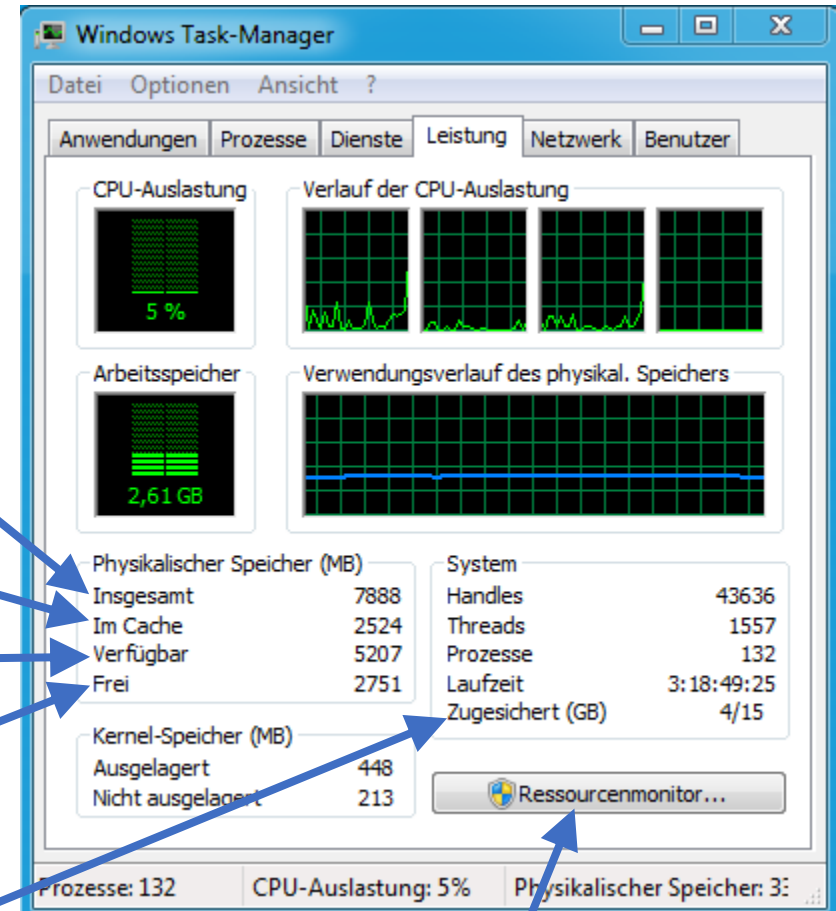
- Seitengröße 4 kiB bis 4 MiB, Seitentabellen pro Prozess
 - 1 GiB für Kernel (für alle Prozesse gleicher phys. Speicher)
 - 3 GiB spezifisch für Prozess
 - Speicherdeskriptor für Segmente (Code, Daten, Stack, Heap, ...)
- 
- Das Diagramm zeigt die Speicherarchitektur für zwei Prozesse, Prozess A und Prozess B. Jeder Prozess hat einen Kernel-adressraum (oben) und einen Benutzeradressraum (unten). Der Kernel-adressraum ist von 0xFFFFF bis 0x00000000. Der Benutzeradressraum ist von 0x00000000 bis 0xFFFFF. Die Größe des Benutzeradressraums ist als TASK_SIZE bezeichnet.
- Paging von Code und Memory Mapped Files direkt aus den jeweiligen (Programm-) Dateien, werden bei Auslagerung nicht kopiert
 - Seiten Laden beim ersten Zugriff (Demand Paging)
 - Ersetzungsstrategie: Page Frame Reclaiming Algorithm ~ Clock Algorithmus
 - kswapd erzeugt bei Speichermangel präventiv freie Seitenrahmen
 - pdflush schreibt präventiv modifizierte Auslagerungskandidaten (Page Buffering)
 - Details abhängig von der Prozessorarchitektur, hier IA-32

P. Mandl: Grundkurs Betriebssysteme, Springer 2020.

Windows Task Manager



- Physikalischer Hauptspeicher (RAM)
- Festplatten-Cache (z. B. Programmcode), kann wenn notwendig gelöscht werden
- sofort (ohne Auslagerung) verfügbar
- enthält keine nützlichen Informationen (Teil von „verfügbar“)
- Virtueller Speicher in Benutzung/Gesamtgröße



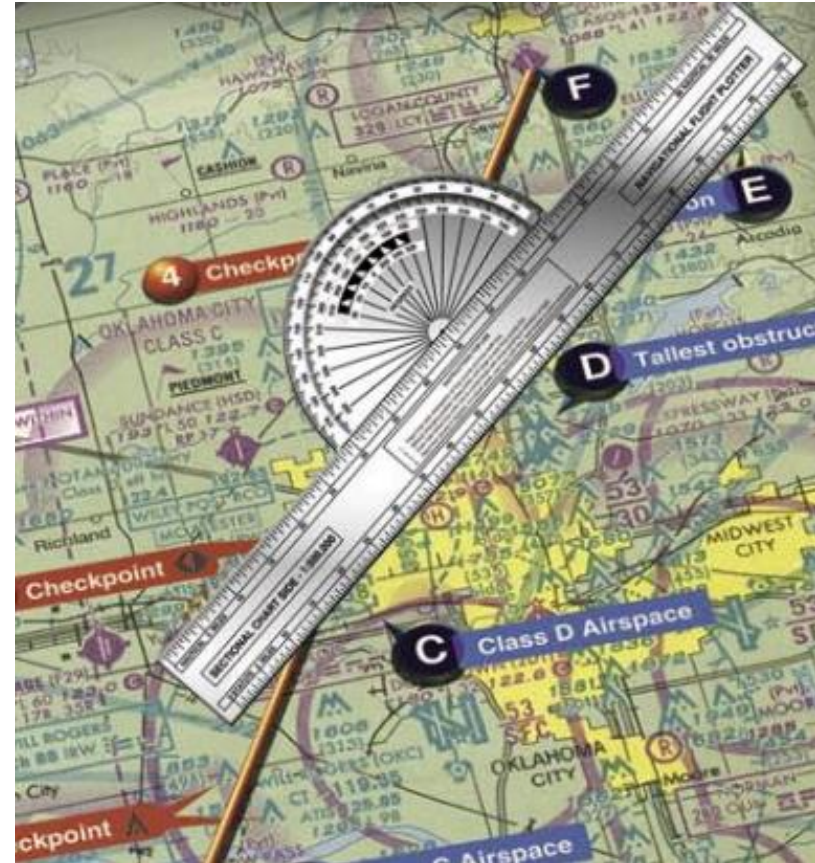
dort „Arbeitsspeicher“ wählen für Infos zu Prozessen

Agenda



Speicherverwaltung

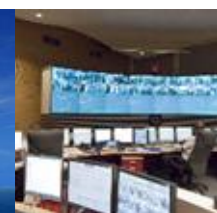
- Problemstellung
- Freier Speicher auf Prozessebene
- Virtueller Speicher
 - Segmentierung
 - Swapping
 - Paging
 - Seitenersetzungsstrategien
- Reales System



Nur zu Lehrzwecken



Betriebssysteme



IA-32 Page Directory Eintrag

Table 4-5. Format of a 32-Bit Page-Directory Entry that References a Page Table

Bit Position(s)	Contents
0 (P)	Present; must be 1 to reference a page table
1 (R/W)	Read/write; if 0, writes may not be allowed to the 4-MByte region controlled by this entry (see Section 4.6)
2 (U/S)	User/supervisor; if 0, user-mode accesses are not allowed to the 4-MByte region controlled by this entry (see Section 4.6)
3 (PWT)	Page-level write-through; indirectly determines the memory type used to access the page table referenced by this entry (see Section 4.9)
4 (PCD)	Page-level cache disable; indirectly determines the memory type used to access the page table referenced by this entry (see Section 4.9)
5 (A)	Accessed; indicates whether this entry has been used for linear-address translation (see Section 4.8)
6	Ignored
7 (PS)	If CR4.PSE = 1, must be 0 (otherwise, this entry maps a 4-MByte page; see Table 4-4); otherwise, ignored
11:8	Ignored
31:12	Physical address of 4-KByte aligned page table referenced by this entry

IA-32 Page Table Eintrag

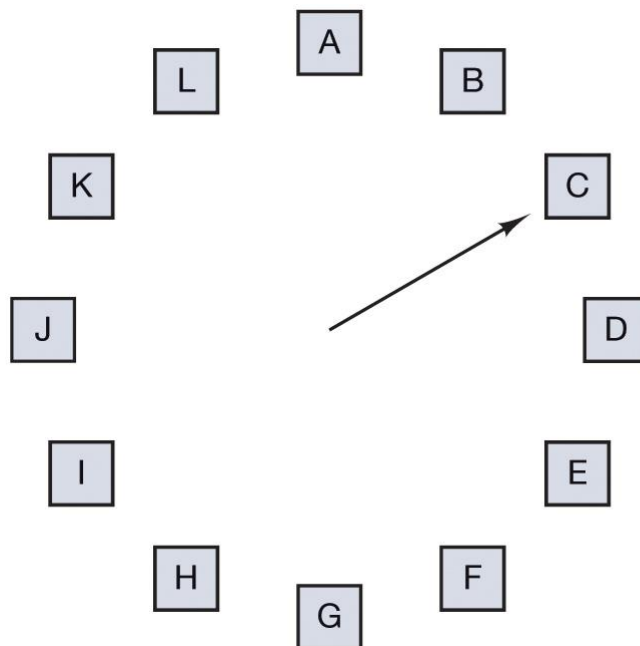
Table 4-6. Format of a 32-Bit Page-Table Entry that Maps a 4-KByte Page

Bit Position(s)	Contents
0 (P)	Present; must be 1 to map a 4-KByte page
1 (R/W)	Read/write; if 0, writes may not be allowed to the 4-KByte page referenced by this entry (see Section 4.6)
2 (U/S)	User/supervisor; if 0, user-mode accesses are not allowed to the 4-KByte page referenced by this entry (see Section 4.6)
3 (PWT)	Page-level write-through; indirectly determines the memory type used to access the 4-KByte page referenced by this entry (see Section 4.9)
4 (PCD)	Page-level cache disable; indirectly determines the memory type used to access the 4-KByte page referenced by this entry (see Section 4.9)
5 (A)	Accessed; indicates whether software has accessed the 4-KByte page referenced by this entry (see Section 4.8)
6 (D)	Dirty; indicates whether software has written to the 4-KByte page referenced by this entry (see Section 4.8)
7 (PAT)	If the PAT is supported, indirectly determines the memory type used to access the 4-KByte page referenced by this entry (see Section 4.9.2); otherwise, reserved (must be 0) ¹
8 (G)	Global; if CR4.PGE = 1, determines whether the translation is global (see Section 4.10); ignored otherwise
11:9	Ignored
31:12	Physical address of the 4-KByte page referenced by this entry



Clock-Algorithmus für Seitenersetzung

- Weiterentwicklung des Second-Chance-Algorithmus
- Verwendet einen *Ringpuffer* anstatt einer FIFO-Struktur
- Der *Zeiger* zeigt auf die jeweils älteste Seite



Wenn ein Seitenfehler auftritt, wird die Seite untersucht, auf die der Zeiger weist. Die Folgeaktion hängt dann vom R-Bit ab:

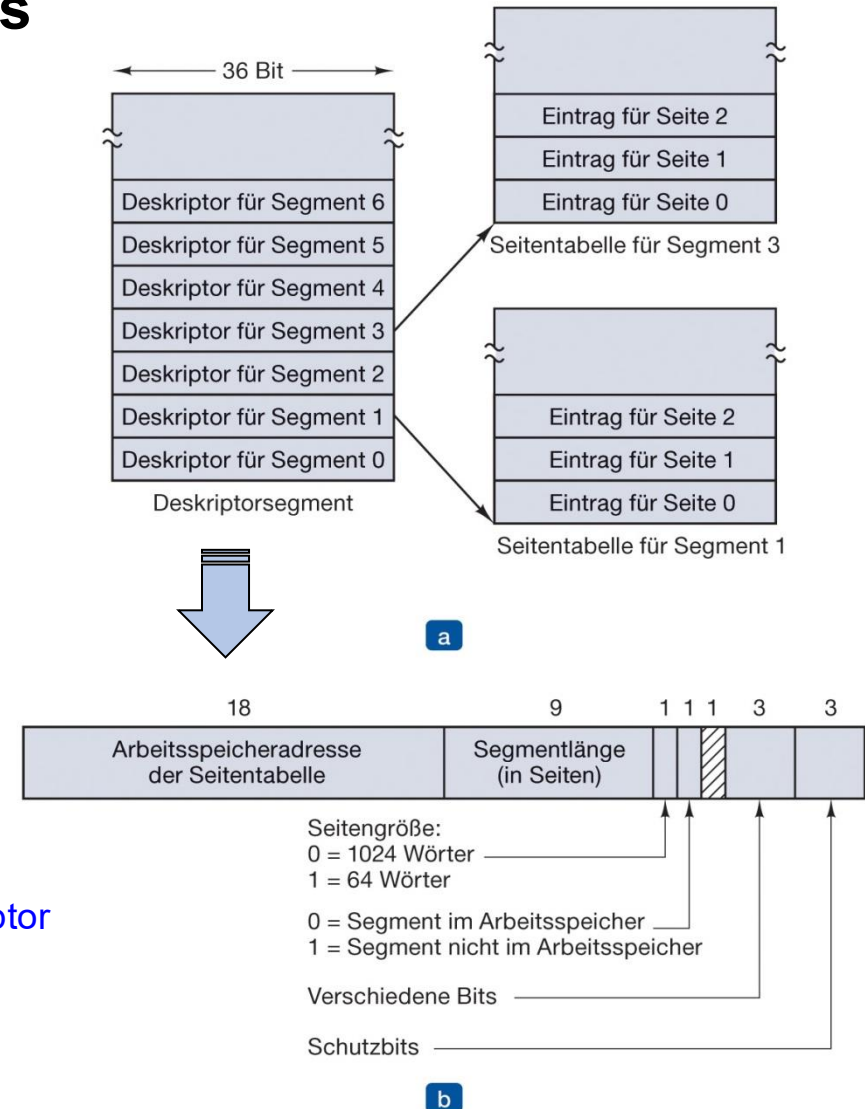
R = 0: verdränge die Seite

R = 1: lösche R und rücke Zeiger weiter

Implementierungsskizze Multics

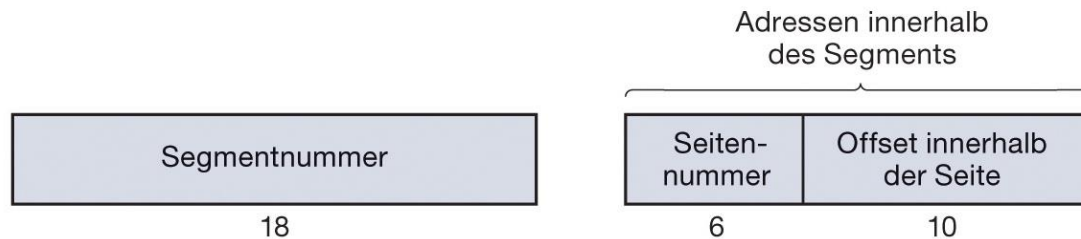
- **Segmentierung mit Paging:**
Segmente werden in Seiten aufgeteilt und seitenweise ein- und ausgelagert.
- **Beispiel: Multics**
 - Jedes Segment bildet einen eigenen virtuellen Adressraum.
 - Pro Segment existiert eine Seitentabelle.

Segmentdeskriptor



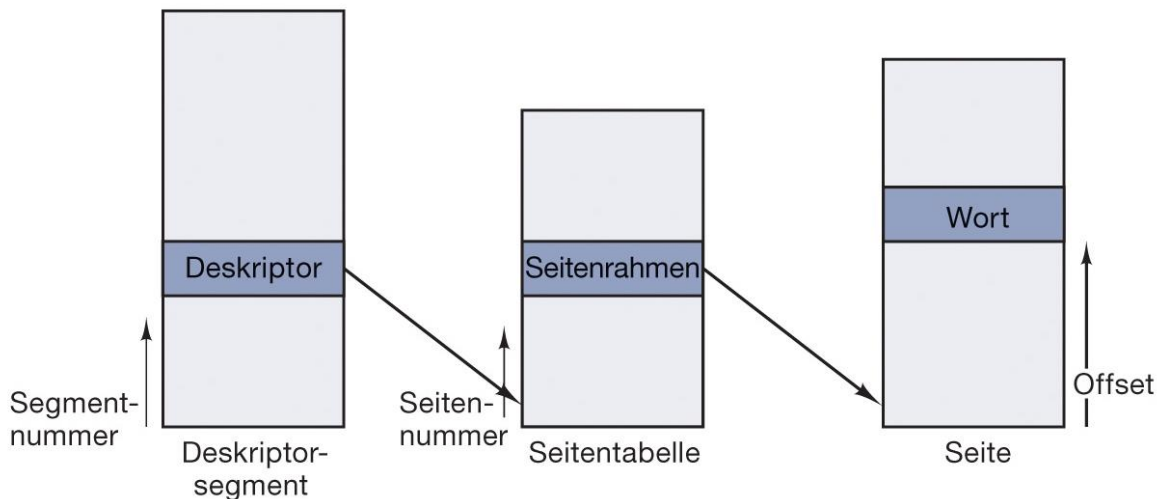
Implementierungsskizze Multics

- Virtuelle 34-Bit-Adresse



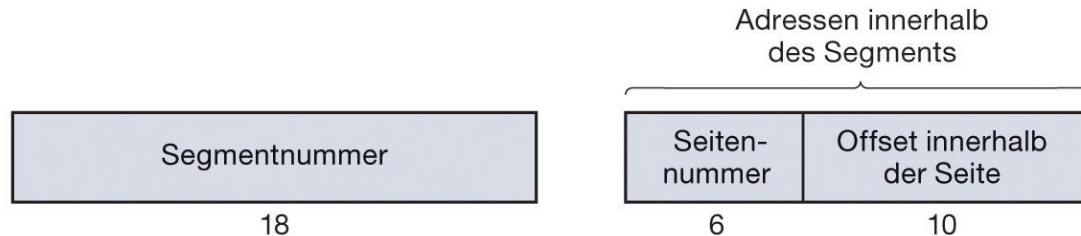
→ 262.144 Segmente zu 64k Worten pro Prozess

- Adressberechnung:



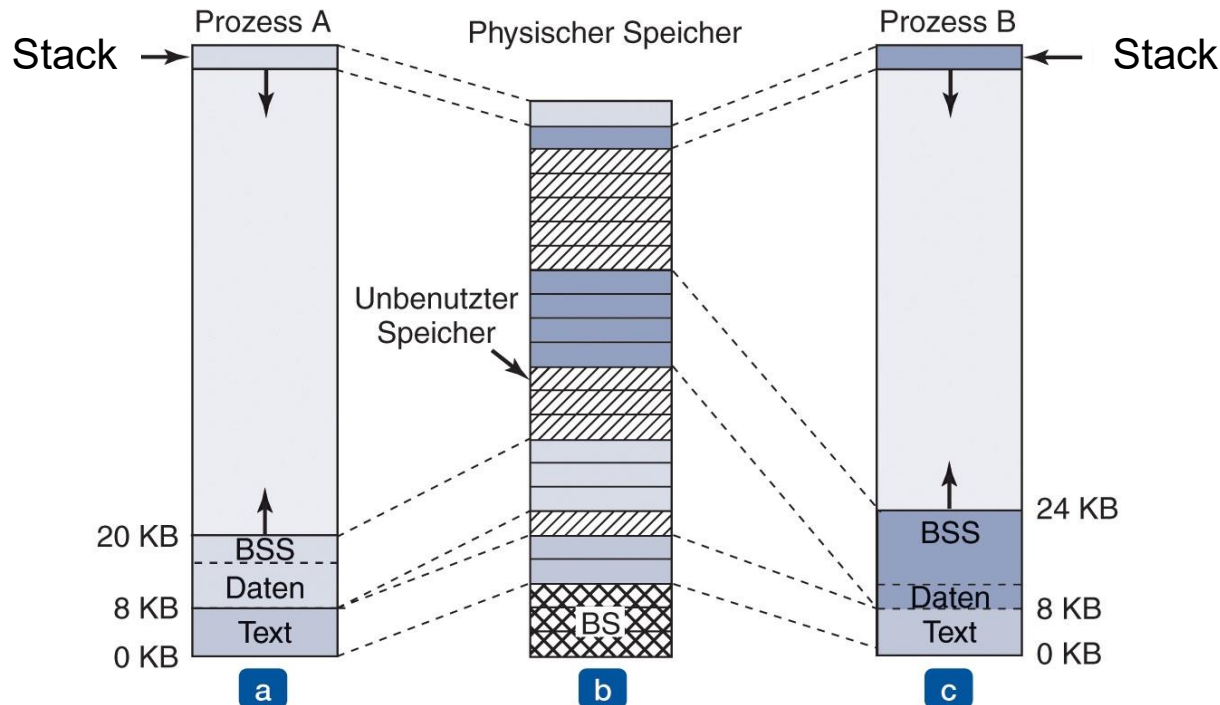
Implementierungsskizze Multics

- Virtuelle 34-Bit-Adresse



- Ablauf bei der Auflösung einer virtuellen Adresse
 1. Über die Segmentnummer wird der Segmentdeskriptor geladen.
 2. Es wird geprüft, ob die Seitentabelle des Segments im Speicher liegt. Wenn nicht, wird ein Segmentfehler ausgelöst. Bei einer Schutzverletzung wird ebenfalls ein Fehler ausgelöst.
 3. Der Seitentabelleneintrag wird geprüft. Liegt die Seite nicht im Speicher, wird ein Seitenfehler ausgelöst.
 4. Der Offset wird zum Seitenanfang hinzuaddiert.
 5. Über die resultierende physikalische Adresse erfolgt der Zugriff.

Speicherverwaltung in Linux



Shared Text-Segment (Maschinencode eines Programms liegt nur einmal im Speicher),

BSS: Block Started by Symbol (nichtinitialisierte Daten, Heap)